

Data Structures and Algorithms Notes

Matthew Du

Last Updated: April 24, 2026

Contents

1	Preliminaries	4
1.1	Asymptotic Notation	4
1.2	Combinatorics	6
1.2.1	Definitions	6
2	Sorting and Searching	8
2.1	Searching	8
2.1.1	Linear Search	8
2.1.2	Binary Search	8
2.1.3	Binary Search Trees	9
2.1.4	Red-Black Trees	9
2.2	Sorting	10
2.2.1	Heap Sort	10
3	Graph Algorithms	12
3.1	Definitions	12
3.1.1	Graphs	12
3.1.2	Trees	13
3.1.3	Graph Representations	13
3.2	Graph Search Algorithms (Unweighted Graphs)	14
3.2.1	Breadth First Search (BFS)	14
3.2.2	Depth-First Search (DFS)	17
3.2.3	Applications	19
3.3	Graph Search Algorithms (Weighted Graphs)	21
3.3.1	Minimum Spanning Trees (MST)	21
3.3.2	Shortest Paths	24
3.3.3	Dijkstra's Algorithm	26
3.3.4	Bellman-Ford	27
3.3.5	Single-Source Shortest Paths in Directed Acyclic Graphs (DAGs)	29
3.3.6	Difference Constraints	30

4	Maximum Flow	33
4.1	Definitions and Flow Diagrams	33
4.1.1	Intuition	33
4.1.2	Formal Definitions	33
4.1.3	More Definitions (Ford Fulkerson)	35
4.2	Ford Fulkerson & Edmond Karp	35
4.2.1	Ford Fulkerson	35
4.2.2	Edmond Karp Algorithm	37
4.3	Maximum Bipartite Matching	37
5	Amortized Analysis	39
5.1	Aggregate Method	39
5.1.1	Example: Stack Operations	39
5.1.2	Example: Binary Counter	40
5.2	Accounting Method	40
5.2.1	Example: Stack	41
5.2.2	Example: Binary Counter	41
5.3	Splay Trees	41
5.3.1	Definitions of Rotations	42
5.3.2	Operations	43
5.3.3	Cost of Splay Operation	43
6	Dynamic Programming	45
6.1	Matrix-Chain Multiplication	45
6.2	Longest Common Subsequence (LCS)	49
7	Greedy Algorithms	52
7.1	Activity-Selection	52
7.2	The Knapsack Problems	54
7.2.1	The Fractional Knapsack Problem	54
7.2.2	The 0–1 Knapsack Problem	54
7.3	Huffman’s Encoding	56
8	Algorithmic Complexity	59
8.1	Definitions	59

8.2	Problem Classes	59
8.2.1	Class P	59
8.2.2	Class NP	60
8.3	NP Completeness	61
8.3.1	NP-Completeness Methodology	62
8.4	Examples	63
8.4.1	Formula Set Satisfiability	63
8.4.2	3-CNF Satisfiability	65
8.4.3	Clique	66
8.4.4	Vertex Cover	67
8.4.5	Traveling Salesman Problem (TSP)	68
8.4.6	Hamiltonian Cycles	69

CHAPTER 1

Preliminaries

1.1 Asymptotic Notation

Big $\mathcal{O}$:

$$f(n) = \mathcal{O}(g(n)) \Leftrightarrow \exists c > 0, \exists n_0 \text{ such that } 0 \leq f(n) \leq c g(n), \forall n \geq n_0$$

Big Ω :

$$f(n) = \Omega(g(n)) \Leftrightarrow \exists c > 0, \exists n_0 \text{ such that } 0 \leq c g(n) \leq f(n), \forall n \geq n_0$$

Big Θ :

$$\begin{aligned} f(n) = \Theta(g(n)) &\Leftrightarrow f(n) = \mathcal{O}(g(n)) \text{ and } f(n) = \Omega(g(n)) \\ &\Leftrightarrow \exists c_1, c_2 > 0, \exists n_0 \text{ such that } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n), \forall n \geq n_0 \end{aligned}$$

Properties

- $n^a \in \mathcal{O}(n^b)$ if and only if $a \leq b$.
- $\log_a n \in \mathcal{O}(\log_b n)$ for all $a, b > 1$.
- $c^n \in \mathcal{O}(d^n)$ if and only if $c \leq d$.
- If $f(n) = \mathcal{O}(g(n))$ and $g(n) = \mathcal{O}(h(n))$, then $f(n) = \mathcal{O}(h(n))$.
- $f(n) = \mathcal{O}(g(n))$ if and only if $g(n) = \Omega(f(n))$.
- Let $f(n) = \mathcal{O}(f'(n))$ and $g(n) = \mathcal{O}(g'(n))$. Then:
 - $f(n) g(n) = \mathcal{O}(f'(n) g'(n))$,
 - $f(n) + g(n) = \mathcal{O}(\max\{f'(n), g'(n)\})$.

Big Theorem: For all positive i, a, b, c ,

$$1 \ll \log^*(n) \ll \log^{(i)} n \ll (\log n)^a \ll \sqrt[n]{n} \ll n \ll n \log n \ll n^{1+b} \ll c^n \ll n!$$

Recursion Tree Method: The **recursion tree method** helps to find a good working guess for solving recurrences using the substitution method.

- The **longest path** in the tree gives an **upper bound** on $T(n)$.
- The **shortest path** in the tree gives a **lower bound** on $T(n)$.

Usually, the total cost of the recurrence is approximated as

$$\text{Total cost} \approx \text{height of the tree} \times \text{cost per level}.$$

More precisely, if $T(n)$ is represented by a recursion tree, the total cost is

$$T(n) = \sum_{\ell=0}^h \text{cost at level } \ell,$$

where h is the height of the tree.

Note: If you use the recursion tree to find an asymptotic bound, you **must** verify it using the substitution method.

Master Theorem: Let $a > 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence

$$T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

where n/b is interpreted as either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$. Then $T(n)$ has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$, then

$$T(n) = O(n^{\log_b a})$$

2. If $f(n) = \Theta(n^{\log_b a})$, then

$$T(n) = O(n^{\log_b a} \log n).$$

3. If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some constant $\varepsilon > 0$, and if

$$af\left(\frac{n}{b}\right) \leq cf(n)$$

for some constant $c < 1$ and all sufficiently large n , then

$$T(n) = O(f(n)).$$

Akra-Bazzi Theorem: Consider the recurrence

$$T(n) = \sum_{i=1}^k a_i T(b_i n) + f(n),$$

where $a_i > 0$, $0 < b_i < 1$, and $f(n)$ is a nonnegative function. Assume $T(n)$ is defined for all sufficiently large n .

Let p be the unique real number satisfying

$$\sum_{i=1}^k a_i b_i^p = 1.$$

(Such a p always exists and is unique.)

Then the solution to the recurrence is

$$T(n) = \Theta\left(n^p \left(1 + \int_1^n \frac{f(x)}{x^{p+1}} dx\right)\right).$$

1.2 Combinatorics

Suppose event A can happen in n ways and event B can happen in m ways.

- **Rule of Product:** If event A happens first and event B happens after A , then the pair (A, B) can occur in nm ways.
- **Rule of Sum:** If either event A or event B can occur (but not both), then there are $n + m$ possible outcomes.

Permutations

The number of permutations of r objects chosen from n distinct objects is

$$P(n, r) = n(n-1) \cdots (n-r+1) = \frac{n!}{(n-r)!}.$$

This is the number of ways to arrange r out of n distinct objects when order is important.

Permutations with Repetition: If not all objects are distinct, suppose there are t types of objects, with q_i objects of the i th type, where $q_1 + q_2 + \cdots + q_t = n$, then the number of distinct arrangements of the n objects is

$$\frac{n!}{q_1! q_2! \cdots q_t!}.$$

Combinations

The number of combinations of r objects chosen from n distinct objects is

$$C(n, r) = \binom{n}{r} = \frac{n!}{r!(n-r)!}.$$

This gives the number of ways to choose a set of r objects from n objects when order does not matter.

1.2.1 Definitions

Log Star:

$$\lg^* n = \min\{i \geq 0 : \lg^{(i)} n \leq 1\}$$

Arithmetic Series:

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}$$

Sums of Squares:

$$\sum_{k=1}^n k^2 = 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

Sums of Cubes:

$$\sum_{k=1}^n k^3 = 1^3 + 2^3 + \cdots + n^3 = \left[\frac{n(n+1)}{2} \right]^2$$

Geometric Series: For $|x| < 1$,

$$\sum_{k=0}^n x^k = \frac{x^{n+1} - 1}{x - 1}$$

When this is an infinite sum,

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1 - x}$$

Harmonic Series:

$$\sum_{k=1}^n \frac{1}{k} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \ln n + \mathcal{O}(1)$$

Telescopic Series:

$$\sum_{k=1}^n (b_k - b_{k+1}) = b_1 - b_{n+1}$$

CHAPTER 2

Sorting and Searching

2.1 Searching

This section describes the algorithms used for searching of items in Arrays

Summary:

	Best case	Average Case	Worst Case	Space
Linear Search	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Binary Search	$\mathcal{O}(1)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$

Table 2.1: Summary of Performance of Searching Algorithms

2.1.1 Linear Search

Probably the most basic list search algorithm, we iterate through each list one item at a time and return the index of the item.

Algorithm 1 LINEARSEARCH(L, T)

```
1: for  $0 \leq i < |L|$  do  
2:   if  $L[i] = T$  then  
3:     return  $i$   
4:   end if  
5: end for
```

2.1.2 Binary Search

This algorithm assumes a sorted list of n items, and divides the search space in half, similar to a divide and conquer algorithm.

Algorithm 2 BINARYSEARCH($A[1 \dots n], x$)

```
1: low  $\leftarrow 1$ 
2: high  $\leftarrow n$ 
3: while low  $\leq$  high do
4:   mid  $\leftarrow \left\lfloor \frac{\text{low} + \text{high}}{2} \right\rfloor$ 
5:   if  $A[\text{mid}] = x$  then
6:     return mid
7:   else if  $A[\text{mid}] < x$  then
8:     low  $\leftarrow$  mid + 1
9:   else
10:    high  $\leftarrow$  mid - 1
11:  end if
12: end while
13: return NIL
```

2.1.3 Binary Search Trees

Binary-Search-Tree property: The keys in a binary search tree are always stored in such a way as to satisfy the BST property:

Let x be a node in a binary search tree. If y is a node in the left subtree of x , then $y.\text{key} \leq x.\text{key}$. If y is a node in the right subtree of x , then $y.\text{key} \geq x.\text{key}$.

Summary of Performance:

Operation	Average Case	Worst Case
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$

Table 2.2: Asymptotic performance of operations in a Binary Search Tree (BST).

2.1.4 Red-Black Trees

A red-black tree is a binary tree that satisfies the following red-black properties:

1. Every node is either red or black.
2. The root is black.
3. Every leaf (NIL) is black.
4. If a node is red, then both its children are black.
5. For each node, all simple paths from the node to descendant leaves contain the same number of black nodes.

Theorem (Lemma 13.1): A red-black tree with n internal nodes has height at most $2 \lg(n + 1)$.

Rotations

The key operation for red black trees is the rotation operation.

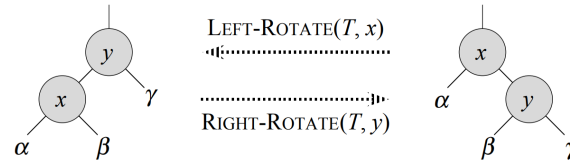


Figure 2.1: Red Black Tree Rotations

Summary of Performance:

Operation	Average Case	Worst Case
Search	$O(\log n)$	$O(\log n)$
Insert	$O(\log n)$	$O(\log n)$
Delete	$O(\log n)$	$O(\log n)$

Table 2.3: Asymptotic performance of operations in a Red-Black Tree (RB-Tree).

2.2 Sorting

Algorithm	Best Case	Average Case	Worst Case
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Randomized QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
HeapSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$
Radix Sort	$O(n \cdot k)$	$O(n \cdot k)$	$O(n \cdot k)$

Table 2.4: Time complexity of common sorting algorithms.

2.2.1 Heap Sort

A **(binary) heap** data structure is an array object that we can view as a nearly complete binary tree. It consists of the **heap property**

It satisfies the heap property, which are defined as such for max and min heaps respectively,

- **Max-Heap:** $A[\text{PARENT}(i)] \leq A[i]$
- **Min-Heap:** $A[\text{PARENT}(i)] \geq A[i]$

Summary of Performance:

Operation	Best Case	Average Case	Worst Case
Search	$O(n)$	$O(n)$	$O(n)$
Insert	$O(1)$	$O(\log n)$	$O(\log n)$
Delete (extract-max/min)	$O(\log n)$	$O(\log n)$	$O(\log n)$
Merge	$O(n + m)$	$O(n + m)$	$O(n + m)$
MakeHeap	$O(n)$	$O(n)$	$O(n)$
Max-Heapify	$O(\log n)$	$O(\log n)$	$O(\log n)$

Table 2.5: Asymptotic performance of operations in a Binary Heap.

CHAPTER 3

Graph Algorithms

3.1 Definitions

3.1.1 Graphs

Graph: A **graph** (G) is a pair of sets $G = (V, E)$, which are defined as follows:

- V is a set of objects; each element of V is called a vertex of the graph.
- E is pairs of objects from V , where each pair (u, v) is a set consisting of two distinct vertices – i.e., $u, v \in V$ and $u \neq v$ – and is called an **edge** of the graph.

In an **undirected graph**, $G = (V, E)$, the edge set E consists of unordered pairs of vertices, while in a **directed graph**, vertices are ordered.

Adjacent Vertices: If (u, v) is an edge in a graph G , we say that vertex v is **adjacent** to vertex u .

Path: Let $G = (V, E)$ and let $u, u' \in V$. A path between u and u' is a sequence of distinct vertices $v_0, v_1, v_2, \dots, v_k \in V$ which satisfy the following properties:

- $v_0 = u$ and $v_k = u'$. (The endpoints of the path are u and u').
- Each consecutive pair of vertices are adjacent. (So v_0 and v_1 are adjacent, and so are v_1 and v_2 , v_2 and v_3 , etc.)

The **length** of the path is the number of edges in the path.

Degree: Let $G = (V, E)$, and let $v \in V$. We say that the degree of v , relationship, denoted $d(v)$, is its number of neighbours, or equivalently, how many edges v is a part of.

Connected: We say that u and u' are connected if and only if there exists a path between u and u' . Because we allow zero-length paths, a vertex is always connected to itself. This definition is existentially quantified; there could be more than one path between u and u' .

Cycles: Let $G = (V, E)$ be a graph. A cycle in G is a sequence of vertices $v_0, \dots, v_k$ such that $v_0 = v_k$ and contains at least one edge.

Edge-Vertices Relationship: Let $G = (V, E)$ be a graph. Then,

$$|E| \leq \frac{|V|(|V| - 1)}{2}$$

Complete Graph: A graph is complete if all vertices are connected to all edges. Mathematically,

it satisfies the relationship,

$$|E| = \frac{|V|(|V| - 1)}{2}$$

3.1.2 Trees

Tree: A tree is a graph that is connected and has no cycles.

Let $G = (V, E)$ be a non-empty rooted tree, and $r \in V$ be the root of the tree. Let $v \in V$ be an arbitrary vertex (including, but not limited to, r itself).

- The **parent** of v is its neighbour which is closer to r than v is. A child of v is any of its other neighbours (which are further from r than v is).
- An **ancestor** of v is any vertex on the path between r and v , not including v itself. (Equivalently, an ancestor of v is its parent, its parent's parent, its parent's parent's parent, etc.)
- A **descendant** of v is any vertex w such that v is on the path between r and w . (Equivalently, a descendant of v is its child, its child's child, its child's child's child, etc.)
- A **leaf** of a rooted tree is any vertex which has no children

Height of a Tree: The height of a non-empty rooted tree is the length of the longest path between the root and a leaf.

Depth of a Tree: The depth of x in T is the length of the simple path from the root r to a node x .

3.1.3 Graph Representations

Adjacency lists: The adjacency-list representation of a graph $G = (V, E)$ consists of an array Adj of $|V|$ lists, one for each vertex in V . For each $u \in V$, the adjacency list $\text{Adj}(u)$ contains all the vertices v such that there is an edge $(u, v) \in E$.

So for the graph below,

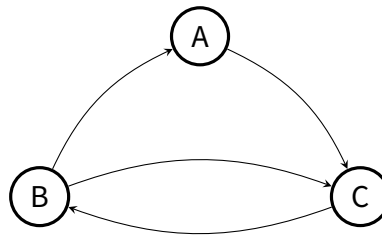


Figure 3.1: Basic Graph

The adjacency list is:

- $\text{Adj}[A] = \{C\}$

- $\text{Adj}[B] = \{A, C\}$
- $\text{Adj}[C] = \{B\}$

Observe this representation requires $\Theta(V + E)$ space.

Adjacency-Matrix: The adjacency-matrix representation of a graph $G = (V, E)$ assumes that the vertices are numbered $1, 2, \dots, |V|$ in some arbitrary manner. Then the adjacency-matrix representation of a graph G consists of a $|V| \times |V|$ matrix $A = (a_{ij})$ such that

$$a_{ij} = \begin{cases} 1 & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}$$

For the graph in Figure 3.1, the adjacency matrix would be,

	A	B	C
A	0	0	1
B	1	0	1
C	0	1	0

Table 3.1: Adjacency Matrix

3.2 Graph Search Algorithms (Unweighted Graphs)

In this section, we will present the basic algorithms used to search for items in graphs.

As a convention, we define the following colours when referring to nodes:

- **White:** An unvisited node
- **Gray:** A node that has been visited, but has not been finished
- **Black:** A node that has been finished; all of the adjacent nodes have been visited.

For a source node s , we also define the functions:

- $d[u]$: The distance from s to u
- $\pi[u]$: The predecessor of u on the path from s to u

3.2.1 Breadth First Search (BFS)

Definition

Given a graph $G = (V, E)$ and a distinguished source vertex s , the breadth-first search systematically explores the edges of G to “discover” every vertex that is reachable from s .

Input: A graph $G = (V, E)$, with a source node s

Output: Two things:

- $d[u]$ = distance from s to u
- $\pi[u]$ = the predecessor of u on the path from s to u

The breadth-first search algorithm works as the following:

1. We initialize all nodes as WHITE and has a distance of ∞ , as we have not visited any of them yet. Their predecessor is NIL as we do not know who they are.
2. Starting at s , s is enqueued and colour set to gray.
3. The first is dequeued, and its neighbours B and D are stored in the queue.
4. While there are items in the queue, repeat:
 - (a) Dequeue the first item in the queue. Call it v .
 - (b) For every node adjacent to v ,
 - i. If it is not white, skip.
 - ii. If it is white, change it to gray and add that node to the queue.
 - (c) Set the colour of u to black, as it is done

Here is an example of the algorithm in action.

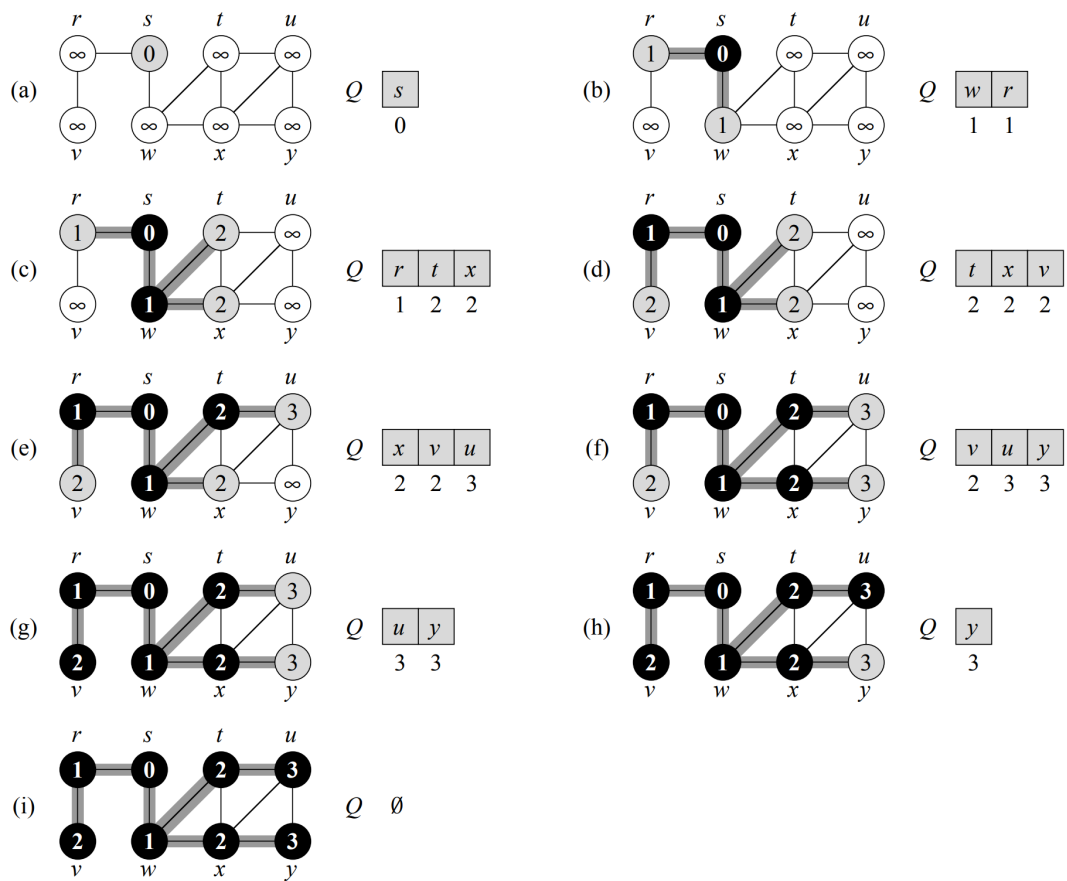
The pseudocode is presented as such:

Algorithm 3 Breadth-First Search (BFS)

```

1: procedure BFS( $G, s$ )
2:   for each vertex  $u \in G.V$  do
3:      $u.color \leftarrow \text{WHITE}$ 
4:      $u.d \leftarrow \infty$ 
5:      $u.\pi \leftarrow \text{NIL}$ 
6:   end for
7:    $s.color \leftarrow \text{GRAY}$ 
8:    $s.d \leftarrow 0$ 
9:    $s.\pi \leftarrow \text{NIL}$ 
10:   $Q \leftarrow \emptyset$ 
11:  ENQUEUE( $Q, s$ )
12:  while  $Q \neq \emptyset$  do
13:     $u \leftarrow \text{DEQUEUE}(Q)$ 
14:    for each  $v \in G.Adj[u]$  do
15:      if  $v.color = \text{WHITE}$  then
16:         $v.color \leftarrow \text{GRAY}$ 
17:         $v.d \leftarrow u.d + 1$ 
18:         $v.\pi \leftarrow u$ 
19:        ENQUEUE( $Q, v$ )
20:      end if
21:    end for
22:     $u.color \leftarrow \text{BLACK}$ 
23:  end while
24: end procedure

```



][H]

Figure 3.2: Breadth First Search Example

By observation, BFS searches every node. Thus, its runtime is $\mathcal{O}(V + E)$.

Application: Shortest Path

3.2.2 Depth-First Search (DFS)

This algorithm recursively explore graph, backtracking as necessary.

It works as follows:

1. Initialize each node as WHITE
2. For each node, recursively visit each of its edges until you cannot go anymore.
3. Mark each edge you visit as gray, and when you cannot go anymore, mark it as black.
4. Repeat until all nodes are visited.

Here is an example:

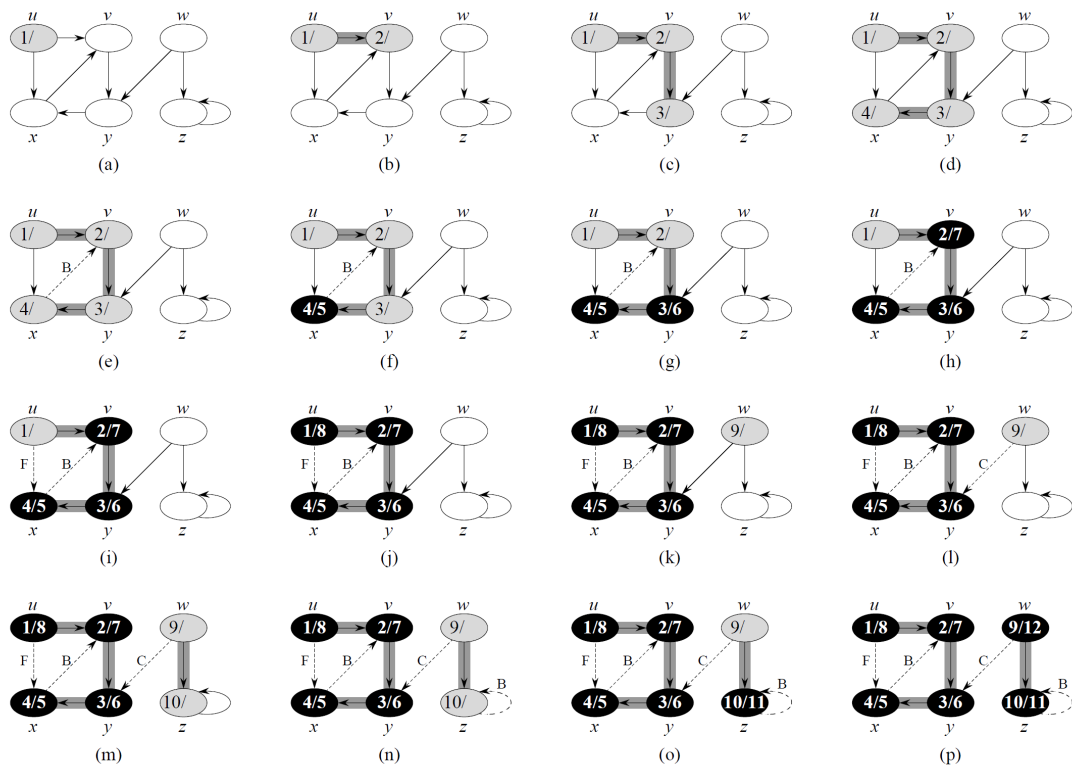


Figure 3.3: Depth First Search Example

The pseudocode is:

Algorithm 4 Depth-First Search

```
1: procedure DFS( $G$ )
2:   for each vertex  $u \in G.V$  do
3:      $u.color \leftarrow \text{WHITE}$ 
4:      $u.\pi \leftarrow \text{NIL}$ 
5:   end for
6:    $time \leftarrow 0$ 
7:   for each vertex  $u \in G.V$  do
8:     if  $u.color = \text{WHITE}$  then
9:       DFS-VISIT( $G, u$ )
10:    end if
11:  end for
12: end procedure
13:
14: procedure DFS-VISIT( $G, u$ )
15:    $time \leftarrow time + 1$ 
16:    $u.d \leftarrow time$ 
17:    $u.color \leftarrow \text{GRAY}$ 
18:   for each vertex  $v \in G.Adj[u]$  do
19:     if  $v.color = \text{WHITE}$  then
20:        $v.\pi \leftarrow u$ 
21:       DFS-VISIT( $G, v$ )
22:     end if
23:   end for
24:    $u.color \leftarrow \text{BLACK}$ 
25:    $time \leftarrow time + 1$ 
26:    $u.f \leftarrow time$ 
27: end procedure
```

By observation, every node is traversed once and DFS-VISIT is called once per node. So the runtime is $\Theta(V + E)$. Although DFS and BFS have similar asymptotic performance, they are used very differently. BFS gives the shortest path, but DFS does not.

Properties

White-Path Theorem (Theorem 22.9): In a depth-first forest of a (directed or undirected) graph $G = (V, E)$ vertex v is a descendant of vertex u if and only if at the time $u.d$ that the search discovers u , there is a path from u to v consisting entirely of white vertices.

Parenthesis Theorem (Theorem 22.7): In any depth-first search of a (directed or undirected) graph $G = (V, E)$, for any two vertices u and v , exactly one of the following three conditions holds:

- the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are entirely disjoint, and neither u nor v is a descendant of the other in the depth-first forest. In other words, intervals do not overlap at all

and DFS finished one vertex completely before it ever started the other, so they are not on the same DFS branch.

- the interval $[u.d, u.f]$ is contained entirely within the interval $[v.d, v.f]$, and u is a descendant of v in a depth-first tree. In other words, u 's interval is completely inside v 's, so DFS discovered u while exploring v , hence u is a descendant of v .
- the interval $[v.d, v.f]$ is contained entirely within the interval $[u.d, u.f]$, and v is a descendant of u in a depth-first tree. In other words, v 's interval is completely inside u 's DFS discovered v while exploring u , so v is a descendant of u .

Note that what cannot happen is,

$$u.d < v.d < u.f < v.f$$

Edge Classification

Tree edges: Edge (u, v) is a tree edge if v was first discovered by exploring edge (u, v) . In other words, a tree edge is an edge that leads to an new vertex. These edges form a tree, which we call the **Depth First Tree**.

Within this “tree”, there can be 3 other types of edges.

Forward edges: Non-tree edges (u, v) connecting a vertex u to a descendant in a depth-first tree.

Back edges: Edges (u, v) connecting a vertex u to an ancestor in a depth-first tree. We consider self-loops, which may occur in directed graphs, to be back edges.

Cross edges: All other edges. They can go between vertices in the same depth-first tree, as long as one vertex is not an ancestor of the other, or they can go between vertices in different depth-first trees.

3.2.3 Applications

In this section, various applications of Unweighted Graph Algorithms are presented

Cycle Detection

For any graph G , a cycle exists iff DFS has a back edge.

Proof. Let $G = (V, E)$.

$(\Leftarrow)$ By definition, a back edge creates a cycle.

$(\Rightarrow)$ If there is a cycle, assume v_0 and v_k are the first and last vertex visited by the depth first search. I claim that (v_0, v_k) is a back edge.

By the parenthesis theorem, we can deduce that that v_i will be visited before we finish v_{i-1} . Thus, by induction, we can deduce we will visit v_k will be visited before v_0 is finished. Thus, (v_0, v_k) is a back edge. ■

Topological Sort

A topological sort of a DAG $G = (V, E)$ is a linear ordering of all its vertices such that if G contains an edge (u, v) , then u appears before v in the ordering. (If the graph contains a cycle, then no linear ordering is possible). Informally, a topological ordering arranges the vertices along a horizontal line so that all edges point from left to right.

The algorithm is presented below,

Algorithm 5 TOPOLOGICAL_SORT

```
1: procedure TOPOLOGICAL_SORT( $V$ ,)
2:   Call DFS( $V, E$ )
3:   Output vertices in decreasing order of finishing times.
4: end procedure
```

The time complexity is the same as DFS, so $\Theta(V + E)$.

Proof of Correctness:

Proof. Assume that u starts before v . We want to show $v.f < u.f$.

Assume u is a gray node (it has been visited, but has not finished).

Case 1: Can v be a gray node? Observe that if this were to be the case, v would be an ancestor of u . However, this means that there is a back edge, making this a cycle, which is not allowed. Thus, this is not possible.

Case 2: Can v be a white node? Observe if this is the case, v becomes a descendant of u . By the parenthesis theorem, this means, $v.f < u.f$. So this is possible

Case 3: Can v be a black node? Observe this means that v is finished. Hence, since u is not finished, that means $v.f < u.f$. So this is possible ■

Strongly Connected Components (SCC)

A **strongly connected component** of a directed graph $G = (V, E)$ is a maximal set of vertices $C \subseteq V$ such that for every pair of vertices u and v in C , we have both $u \rightsquigarrow v$ and $v \rightsquigarrow u$; that is, vertices u and v are reachable from each other. In other words, a strongly connected component is a group of vertices in a directed graph where you can get from any one of them to any other by following the arrow directions.

The algorithm operates on the transpose of G . This is defined as $G^\top = (V, E^\top)$, where $E^\top = \{(u, v) \mid (v, u) \in E\}$. That is, $E^\top$ consists of the edges of G with their directions reversed.

By observation, the SCC of a transpose is the same as the original. Thus, we can compute the strongly connected components of G by first performing a depth-first search on G to determine finishing times, and then performing depth-first searches on $G^\top$ in decreasing order of those finishing times.

The algorithm is as follows:

Algorithm 6 STRONGLY-CONNECTED-COMPONENTS(G)

- 1: call DFS(G) to compute finishing times $u.f$ for each vertex u
 - 2: compute $G^\top$
 - 3: call DFS($G^\top$), but in the main loop of DFS, consider the vertices in order of decreasing $u.f$ (as computed in line 1)
 - 4: output the vertices of each tree in the depth-first forest formed in line 3 as a separate strongly connected component
-

The performance of the algorithm is $\Theta(V + E)$.

3.3 Graph Search Algorithms (Weighted Graphs)

In this section, we want to analyze graphs with weights. This is a graph $G = (V, E)$ together with a function $w : E \rightarrow \mathbb{R}$ that assigns a real weight $w(e)$ to each edge e , which may be positive, negative, or zero.

3.3.1 Minimum Spanning Trees (MST)

Suppose we are given a connected, undirected, weighted graph. A minimum spanning tree of G is the spanning tree $T \subseteq E$ that connects all of the vertices and whose total weight and minimizes the function,

$$w(T) = \sum_{e \in T} w(e)$$

Properties of MSTs:

- An MST has exactly $|V| - 1$ edges (since it connects each node)
- An MST has no cycles (otherwise it is not a tree)
- An MST may not be unique (may have many different possible minimum spanning trees)

Safe Edge: A safe edge is an edge you can add to your current set of edges and still be guaranteed that all those edges belong to the same minimum spanning tree. Formally, given a set of edges $A \subseteq \text{MST}$, an edge (u, v) is known as a **safe edge** if $A \cup \{(u, v)\}$ is also a subset of the same MST.

Generic MST algorithm:

Algorithm 7 GENERIC-MST(G, w)

- 1: $A \leftarrow \emptyset$
 - 2: **while** A does not form a spanning tree **do**
 - 3: find an edge (u, v) that is safe for A
 - 4: $A \leftarrow A \cup \{(u, v)\}$
 - 5: **end while**
 - 6: **return** A
-

Finding Safe Edges:

Cut: A **cut** $(S, V \setminus S)$ of an undirected graph $G = (V, E)$ is a partition of V .

Crosses: We say that an edge $(u, v) \in E$ crosses the cut $(S, V \setminus S)$ if one of its endpoints is in S and the other is in $V \setminus S$.

Respects: We say that a cut **respects** a set A of edges if no edge in A crosses the cut.

Light Edge: An edge is a **light edge** crossing a cut if its weight is the minimum among all edges crossing the cut. Note that there can be more than one light edge crossing a cut in the case of ties.

Theorem (Theorem 23.1): Let $G = (V, E)$ be a connected, undirected graph with a real-valued weight function w defined on E . Let $A \subseteq E$ be a subset of edges that is included in some minimum spanning tree for G , let $(S, V \setminus S)$ be any cut of G that respects A , and let (u, v) be a **light edge** crossing $(S, V \setminus S)$. Then, the edge (u, v) is **safe** for A .

Proof. Let T be a minimum spanning tree that includes A , and assume that T does not contain the light edge (u, v) , since if it does, we are done. We shall construct another minimum spanning tree T' that includes $A \cup \{(u, v)\}$ by using a cut-and-paste technique, thereby showing that (u, v) is a safe edge for A .

The edge (u, v) forms a cycle with the edges on the simple path p from u to v in T . Since u and v are on opposite sides of the cut $(S, V \setminus S)$, at least one edge in T lies on the simple path p and also crosses the cut. Let (x, y) be any such edge. The edge (x, y) is not in A , because the cut respects A . Since (x, y) is on the unique simple path from u to v in T , removing (x, y) breaks T into two components.

Adding (u, v) reconnects them to form a new spanning tree

$$T' = T \setminus \{(x, y)\} \cup \{(u, v)\}.$$

We next show that T' is a minimum spanning tree. Since (u, v) is a light edge crossing $(S, V \setminus S)$ and (x, y) also crosses this cut, $w(u, v) \leq w(x, y)$. Therefore,

$$w(T') = w(T) - w(x, y) + w(u, v) \leq w(T).$$

But T is a minimum spanning tree, so $w(T) \leq w(T')$; thus, T' must also be a minimum spanning tree.

It remains to show that (u, v) is actually a safe edge for A . We have $A \subseteq T'$, since $A \subseteq T$ and $(x, y) \notin A$; thus, $A \cup \{(u, v)\} \subseteq T'$. Consequently, since T' is a minimum spanning tree, (u, v) is safe for A . ■

Prim's Algorithm

Prim's algorithm grows the MST one vertex at a time:

- Start with any vertex as the MST.

- Repeatedly add the lightest edge that connects a vertex in the MST to a vertex outside the MST.
- Continue until all vertices are included

We keep all the edges adjacent to T in a priority queue. When we pull the minimum-weight edge out of the priority queue, we first check whether both of its endpoints are in T . If not, we add the edge to T and then add the new neighboring edges to the priority queue (Q).

Algorithm 8 MST-PRIM(G, w, r)

```

1: for  $u \in G.V$  do
2:    $u.key \leftarrow \infty$ 
3:    $u.\pi \leftarrow \text{NIL}$ 
4: end for
5:  $r.key \leftarrow 0$ 
6:  $Q \leftarrow G.V$ 
7: while  $Q \neq \emptyset$  do
8:    $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
9:   for  $v \in G.Adj[u]$  do
10:    if  $v \in Q$  and  $w(u, v) < v.key$  then
11:       $v.\pi \leftarrow u$ 
12:       $v.key \leftarrow w(u, v)$ 
13:    end if
14:  end for
15: end while

```

An example of Prim's Algorithm:

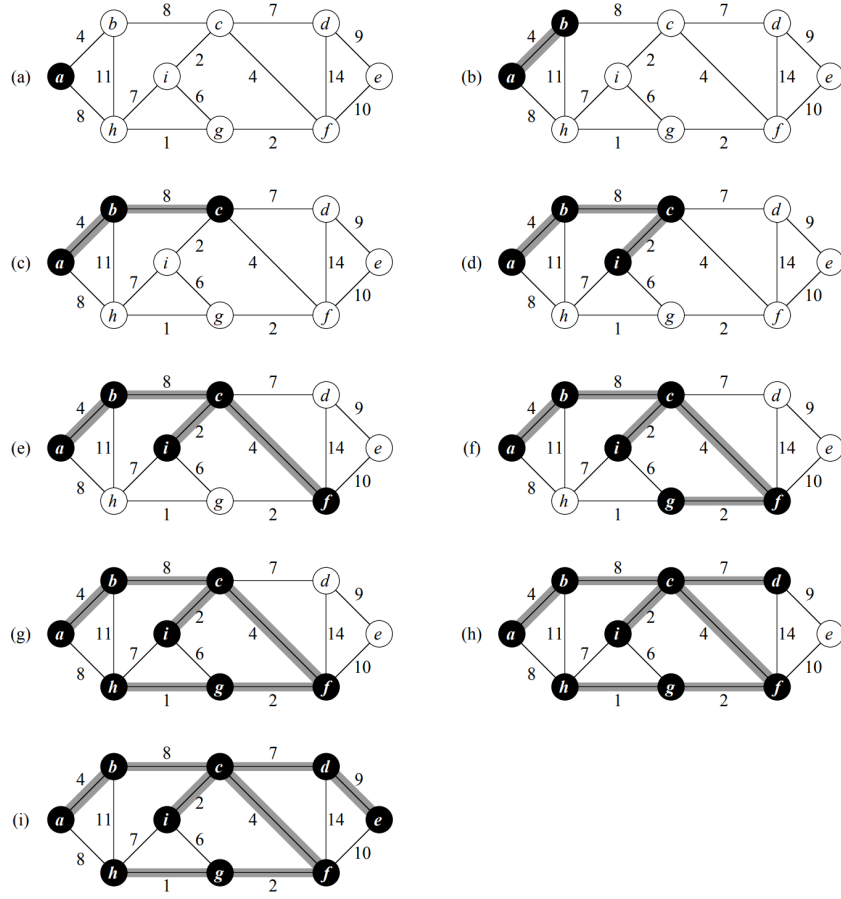


Figure 3.4: Prim's Algorithm Example

The performance of Prim's algorithm is $\mathcal{O}(V \log V + E \log V) = \mathcal{O}(E \log V)$. If we use a Fibonacci Heap to implement a min priority queue, then the performance is $\mathcal{O}(E + V \log V)$.

3.3.2 Shortest Paths

Shortest Path: Given a weighted, directed graph $G = (V, E)$, with weight function $w: E \rightarrow \mathbb{R}$ mapping edges to real-valued weights. The weight $w(p)$ of a path $p = \langle v_0, v_1, \dots, v_k \rangle$ is the sum of the weights of its constituent edges:

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i).$$

We define the shortest-path weight $\delta(u, v)$ from u to v by

$$\delta(u, v) = \begin{cases} \min\{w(p) \mid u \xrightarrow{p} v\} & \text{if there is a path from } u \text{ to } v, \\ \infty & \text{otherwise.} \end{cases}$$

A **shortest path** from vertex u to vertex v is then defined as any path p with weight $w(p) = \delta(u, v)$. Shortest Paths may not be unique

Basic Properties and Assumptions

1. **Negative Weight Edges:** Negative weight edges are OK to have as long as there are no negative weight cycles in G from the source. A negative cycle lets you keep going around the cycle and make the total cost smaller and smaller forever. If that happens, there is no “shortest” path, because you can always get a cheaper one.
2. **Optimal Substructure:** Any subpath of a shortest path is a shortest path itself. For example, if a path from $A \rightarrow B \rightarrow C \rightarrow D$ is the shortest path from A to D , then: $A \rightarrow B$, $B \rightarrow C$, and $C \rightarrow D$ are all shortest paths too.
3. **Cycles:** Cycles-shortest paths cannot contain cycles. Going in a cycle only adds extra cost or waste.

Output: After each shortest path algorithm, we obtain the following. For each vertex $v \in V$,

- $v.d = \delta(s, v)$. Initially, $v.d = \infty$ and $s.d = 0$. At every step, $v.d$ progressively gets smaller but $v.d \geq \delta(s, v)$ always
- $v.\pi$ = predecessor of v on the shortest path from s

Algorithm 9 INITIALIZE-SINGLE-SOURCE(G, s)

```
1: for each vertex  $v \in G.V$  do
2:    $v.d \leftarrow \infty$ 
3:    $v.\pi \leftarrow \text{NIL}$ 
4: end for
5:  $s.d \leftarrow 0$ 
```

Relaxation: The process of relaxing an edge (u, v) consists of testing whether we can improve the shortest path to v found so far by going through u and, if so, updating $v.d$ and $v.\pi$.

Algorithm 10 RELAX(u, v, w)

```
1: if  $v.d > u.d + w(u, v)$  then
2:    $v.d = u.d + w(u, v)$ 
3:    $v.\pi = u$ 
4: end if
```

Some miscellaneous properties:

Triangle inequality (Lemma 24.10). For any edge $(u, v) \in E$, we have

$$\delta(s, v) \leq \delta(s, u) + w(u, v).$$

Upper-bound property (Lemma 24.11). We always have $v.d \geq \delta(s, v)$ for all vertices $v \in V$, and once $v.d$ achieves the value $\delta(s, v)$, it never changes.

No-path property (Corollary 24.12). If there is no path from s to v , then we always have

$$v.d = \delta(s, v) = \infty.$$

Convergence property (Lemma 24.14). If $s \xrightarrow{p} u \rightarrow v$ is a shortest path in G for some $u, v \in V$, and if $u.d = \delta(s, u)$ at any time prior to relaxing edge (u, v) , then $v.d = \delta(s, v)$ at all times afterward.

Path-relaxation property (Lemma 24.15). If $p = \langle v_0, v_1, \dots, v_k \rangle$ is a shortest path from $s = v_0$ to v_k , and we relax the edges of p in the order $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$, then

$$v_k.d = \delta(s, v_k).$$

This property holds regardless of any other relaxation steps that occur, even if they are inter-mixed with relaxations of the edges of p .

Predecessor-subgraph property (Lemma 24.17). Once $v.d = \delta(s, v)$ for all $v \in V$, the predecessor subgraph is a shortest-paths tree rooted at s .

3.3.3 Dijkstra's Algorithm

Dijkstra's algorithm solves the single-source shortest-paths problem on a weighted, directed graph $G = (V, E)$ for the case in which all edge weights are non-negative.

Dijkstra's Algorithm stores two sets:

- S = set of vertices for which the final shortest path weight is determined
- Q = priority queue = $V - S$

Algorithm 11 DIJKSTRA(G, w, s)

```

1: INITIALIZE-SINGLE-SOURCE( $G, s$ )
2:  $S \leftarrow \emptyset$ 
3:  $Q \leftarrow G.V$ 
4: while  $Q \neq \emptyset$  do
5:    $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
6:    $S \leftarrow S \cup \{u\}$ 
7:   for each vertex  $v \in G.Adj[u]$  do
8:     RELAX( $u, v, w$ )
9:   end for
10: end while

```

An example of how Dijkstra's algorithm works is shown below:

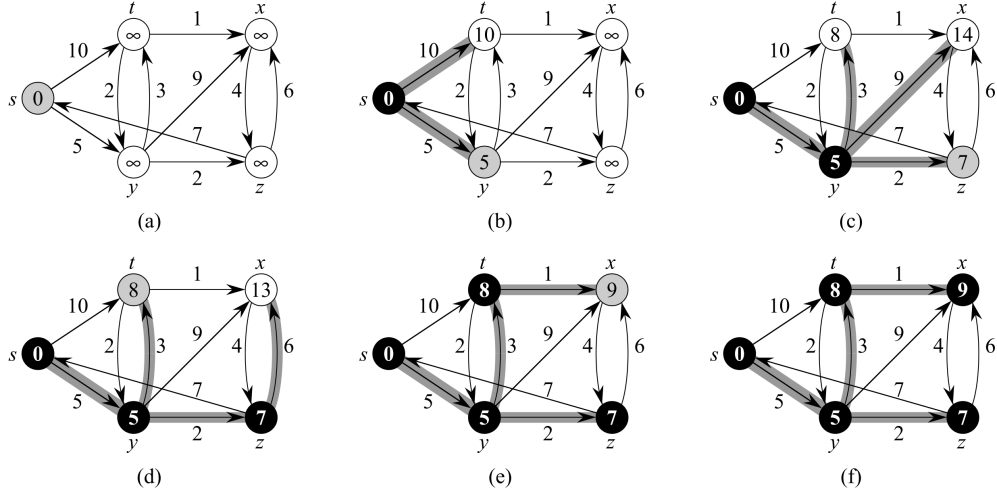


Figure 3.5: Dijkstra's Example

Using a binary heap implementation of a priority queue, the total running time is therefore $\mathcal{O}((V + E) \log V)$, which becomes $\mathcal{O}(E \log V)$ if all vertices are reachable from the source.

3.3.4 Bellman-Ford

The Bellman-Ford algorithm solves the single-source shortest-paths problem in the general case in which edge weights may be negative. Given a weighted, directed graph $G = (V, E)$ with source s and weight function $w: E \rightarrow \mathbb{R}$, the Bellman-Ford algorithm returns a boolean value indicating whether or not there is a negative-weight cycle that is reachable from the source.

Algorithm 12 BELLMAN-FORD(G, w, s)

```

1: INITIALIZE-SINGLE-SOURCE( $G, s$ )
2: for  $i \leftarrow 1$  to  $|G.V| - 1$  do
3:   for edge  $(u, v) \in G.E$  do
4:     RELAX( $u, v, w$ )
5:   end for
6: end for
7: for each edge  $(u, v) \in G.E$  do
8:   if  $v.d > u.d + w(u, v)$  then
9:     return false
10:  end if
11: end for
12: return true

```

The runtime of Bellman Ford is $\mathcal{O}(VE)$

An example of execution is shown below:

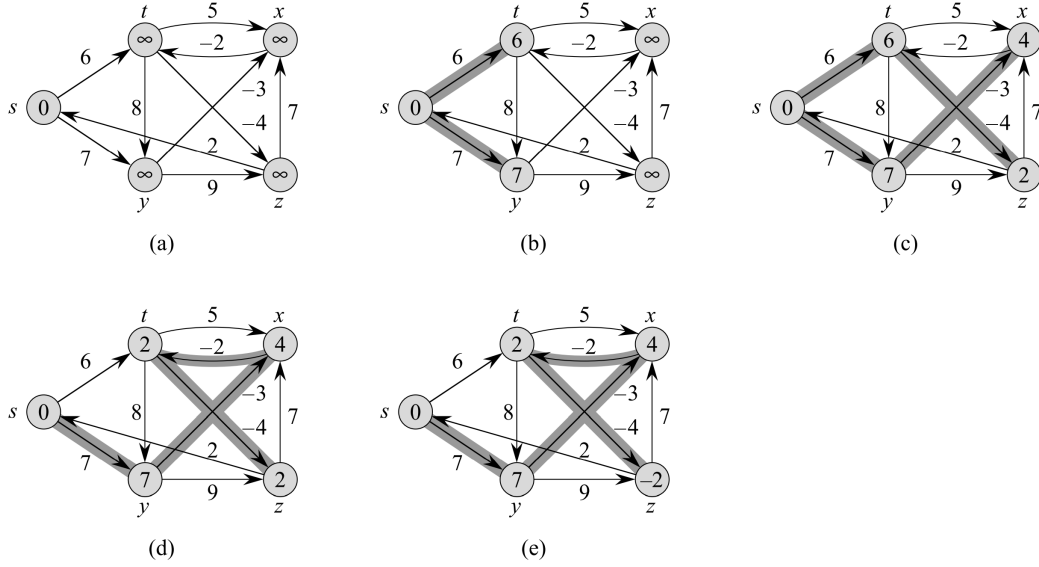


Figure 3.6: Bellman-Ford

Correctness of the Bellman-Ford Algorithm (Theorem 22.4): Let **BELLMAN-FORD** be run on a weighted, directed graph $G = (V, E)$ with source s and weight function $w: E \rightarrow \mathbb{R}$. If G contains no negative-weight cycles that are reachable from s , then the algorithm returns **true**, we have $v.d = \delta(s, v)$ for all vertices $v \in V$, and the predecessor subgraph G_π is a shortest-paths tree rooted at s . If G does contain a negative-weight cycle reachable from s , then the algorithm returns **false**.

Proof. To prove that this is the shortest path, we need to prove,

- $v.d = \delta(s, v)$.
- $v.\pi =$ predecessor of v on the shortest path from s

We consider 2 cases:

- If graph G contains no negative-weight cycles that are reachable from the source s . We first prove the claim that at termination, $v.d = \delta(s, v)$ for all vertices $v \in V$. If vertex v is reachable from s , then this is true by the triangular inequality. If v is not reachable from s , then the claim follows from the no-path property and **BELLMAN-FORD** returns **False**. Thus, the claim is proven.

For the The predecessor-subgraph property, along with the claim, implies that G_π is a shortest-paths tree. Now we use the claim to show that **BELLMAN-FORD** returns **true**. At termination, for all edges $(u, v) \in E$, we have

$$\begin{aligned}
 v.d &= \delta(s, v) \\
 &\leq \delta(s, u) + w(u, v) \quad (\text{by the triangle inequality}) \\
 &= u.d + w(u, v)
 \end{aligned}$$

- Now, suppose that graph G contains a negative-weight cycle that is reachable from the source s ; let this cycle be $c = \langle v_0, v_1, \dots, v_k \rangle$, where $v_0 = v_k$. Then,

$$\sum_{i=1}^k w(v_{i-1}, v_i) < 0$$

Assume for the purpose of contradiction that the Bellman–Ford algorithm returns **true**.

Thus, $v_i.d \leq v_{i-1}.d + w(v_{i-1}, v_i)$ for $i = 1, 2, \dots, k$.

Summing the inequalities around cycle c gives

$$\sum_{i=1}^k v_i.d \leq \sum_{i=1}^k (v_{i-1}.d + w(v_{i-1}, v_i)) = \sum_{i=1}^k v_{i-1}.d + \sum_{i=1}^k w(v_{i-1}, v_i).$$

Since $v_0 = v_k$, each vertex in c appears exactly once in each of the summations $\sum_{i=1}^k v_i.d$ and $\sum_{i=1}^k v_{i-1}.d$, and so

$$\sum_{i=1}^k v_i.d = \sum_{i=1}^k v_{i-1}.d.$$

Moreover, since a path exists for Bellman–Ford if $v_i.d < \infty$, $v_i.d$ is finite for $i = 1, 2, \dots, k$. Thus,

$$0 \leq \sum_{i=1}^k w(v_{i-1}, v_i),$$

which contradicts the initial inequality. We conclude that the Bellman–Ford algorithm returns **true** if graph G contains no negative-weight cycles reachable from the source, and **false** otherwise. ■

3.3.5 Single-Source Shortest Paths in Directed Acyclic Graphs (DAGs)

By relaxing the edges of a weighted DAG (directed acyclic graph) $G = (V, E)$ according to a topological sort of its vertices, we can compute shortest paths from a single source in $\Theta(V + E)$ time. Shortest paths are always well defined in a DAG, since even if there are negative-weight edges, no negative-weight cycles can exist.

Algorithm 13 DAG-SHORTEST-PATHS(G, w, s)

- 1: topologically sort the vertices of G
 - 2: INITIALIZE-SINGLE-SOURCE(G, s)
 - 3: **for** each vertex u , taken in topologically sorted order **do**
 - 4: **for** each vertex $v \in G.Adj[u]$ **do**
 - 5: RELAX(u, v, w)
 - 6: **end for**
 - 7: **end for**
-

An example of execution is found below:

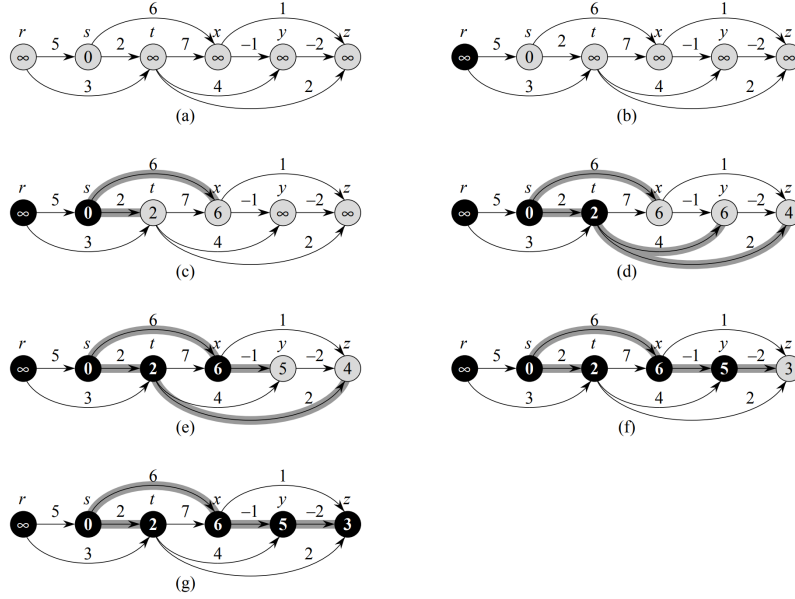


Figure 3.7: Single-Source Shortest Paths in Directed Acyclic Graphs (DAGs)

3.3.6 Difference Constraints

In the general linear-programming problem, we are given an $m \times n$ matrix A , an m -vector $\vec{b}$, and an n -vector $\vec{c}$. We wish to find a vector $\vec{x}$ of n elements that maximizes the objective function

$$\sum_{i=1}^n c_i x_i$$

subject to the m constraints given by $A\vec{x} \leq \vec{b}$.

In a system of **difference constraints**, each row of the linear-programming matrix A contains one 1 and one -1 , and all other entries of A are 0. Thus, the constraints given by $A\vec{x} \leq \vec{b}$ are a set of m difference constraints involving n unknowns, in which each constraint is a simple linear inequality of the form

$$x_j - x_i \leq b_k,$$

where $1 \leq i, j \leq n, i \neq j$, and $1 \leq k \leq m$.

For example, consider the problem of finding a 5-vector $\vec{x} = (x_i)$ that satisfies

$$\begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 & -1 \\ -1 & 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & -1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix} \leq \begin{pmatrix} 0 \\ -1 \\ 1 \\ 5 \\ 4 \\ -1 \\ -3 \\ -3 \end{pmatrix}.$$

This problem is equivalent to finding values for the unknowns x_1, x_2, x_3, x_4, x_5 satisfying the following 8 difference constraints:

$$\begin{aligned} x_1 - x_2 &\leq 0 \\ x_1 - x_5 &\leq -1 \\ x_2 - x_5 &\leq 1 \\ x_3 - x_1 &\leq 5 \\ x_4 - x_1 &\leq 4 \\ x_4 - x_3 &\leq -1 \\ x_5 - x_3 &\leq -3 \\ x_5 - x_4 &\leq -3 \end{aligned}$$

Solution: Build a constraint graph $G = (V, E)$. Each vertex v_i in the graph, for $i = 1, 2, \dots, n$, corresponds to one of the n unknown variables x_i . Each directed edge in the graph corresponds to one of the m inequalities involving two unknowns

Formally, for a system $A\vec{x} \leq \vec{b}$ of difference constraints, the corresponding **constraint graph** is a weighted, directed graph $G = (V, E)$, where

$$V = \{v_0, v_1, \dots, v_n\}$$

and

$$E = \{(v_i, v_j) \mid x_j - x_i \leq b_k \text{ is a constraint}\} \cup \{(v_0, v_1), (v_0, v_2), (v_0, v_3), \dots, (v_0, v_n)\}.$$

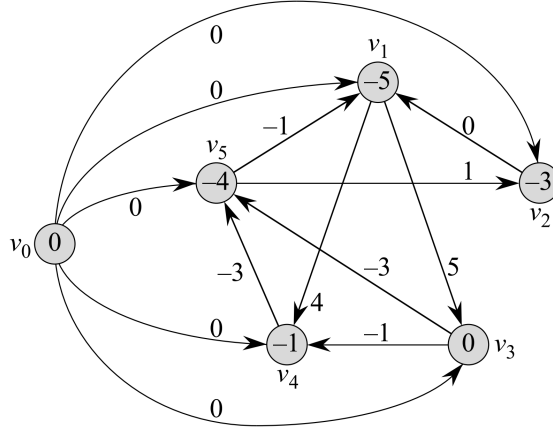


Figure 3.8: Constraint Graph

Theorem (Theorem 24.9): Given a system $A\vec{x} \leq \vec{b}$ of difference constraints, let $G = (V, E)$ be the corresponding constraint graph.

If G contains no negative-weight cycles, then

$$\vec{x} = (\delta(v_0, v_1), \delta(v_0, v_2), \delta(v_0, v_3), \dots, \delta(v_0, v_n))$$

is a feasible solution for the system.

If G contains a negative-weight cycle, then there is no feasible solution for the system.

Algorithm: Using the above theorem, we can use the Bellman-Ford algorithm to solve a system of difference constraints. Because the constraint graph contains edges from the source vertex v_0 to all other vertices, any negative-weight cycle in the constraint graph is reachable from v_0 . If the Bellman-Ford algorithm returns TRUE, then the shortest-path weights give a feasible solution to the system.

The time needed to build the graph is $\mathcal{O}(n + m)$ and the runtime of Bellman-Ford is $\mathcal{O}(EV) = \mathcal{O}((n + 1)(n + m)) = \mathcal{O}(n^2 + nm)$. Thus, the overall runtime is,

$$\mathcal{O}(n^2 + nm)$$

CHAPTER 4

Maximum Flow

4.1 Definitions and Flow Diagrams

4.1.1 Intuition

A flow diagram is a diagram that shows:

1. **Nodes (or vertices):** These are the points or locations in the network.
2. **Edges:** These are the connections between the nodes.
3. **Capacities:** Each edge has a maximum allowed flow.
4. **Flow:** While running the algorithm, you also track how much flow is currently in each edge.

For every node except the source and the sink, the total flow into the node must equal the total flow out of the node.

This is usually written as: flow / capacity, for example 3 / 10.

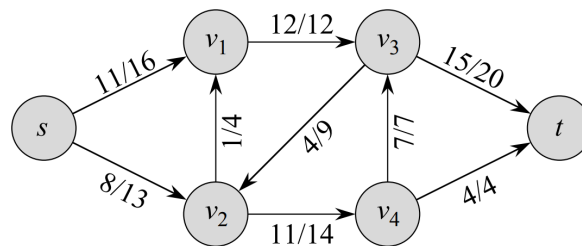


Figure 4.1: Maximum Flow Introduction Diagram

Cut: In a flow network, a cut is a way of splitting the set of all nodes into two groups: One group that contains the source s , and another that contains the sink t

Problem: What is the maximum amount of flow that we can carry from source to sink?

4.1.2 Formal Definitions

Flow Network: A flow network $G = (V, E)$ is a directed graph in which each edge $(u, v) \in E$ has a nonnegative **capacity** $c(u, v) \geq 0$. We further require that if E contains an edge (u, v) ,

then there is no edge (v, u) in the reverse direction. If $(u, v) \notin E$, then for convenience we define $c(u, v) = 0$, and we disallow self-loops.

We distinguish two vertices in a flow network: a **source** s and a **sink** t . For convenience, we assume that each vertex lies on some path from the source to the sink; that is, for each vertex $v \in V$, the flow network contains a path $s \rightsquigarrow v \rightsquigarrow t$. The graph is therefore connected and, since each vertex other than s has at least one entering edge, $|E| \geq |V| - 1$.

Flow: Let $G = (V, E)$ be a flow network with a capacity function c . Let s be the source of the network, and let t be the sink. A flow in G is a real-valued function $f : V \times V \rightarrow \mathbb{R}$ that satisfies the following two properties:

- **Capacity constraint:** For all $u, v \in V$, we require $0 \leq f(u, v) \leq c(u, v)$.
- **Flow conservation:** For all $u \in V \setminus \{s, t\}$, we require

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v).$$

When $(u, v) \notin E$, there can be no flow from u to v , and $f(u, v) = 0$.

We call the nonnegative quantity $f(u, v)$ the flow from vertex u to vertex v . The value $|f|$ of a flow f is defined as

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s),$$

that is, the total flow out of the source minus the flow into the source.

Notation: We define the flow between two sets of vertices X, Y as,

$$f(X, Y) = \sum_{x \in X} \sum_{y \in Y} f(x, y)$$

Lemma (Properties)

1. $f(X, X) = 0$
2. $f(X, Y) = -f(Y, X)$
3. $f(X \cup Y, W) = f(X, W) + f(Y, W)$ if $X \cap Y = \emptyset$
4. $f(W, X \cup Y) = f(W, X) + f(W, Y)$ if $X \cap Y = \emptyset$

Skew Symmetry: For any two nodes $u, v \in V$, define $f(u, v) = -f(v, u)$. Note: If an edge does not exist, we assume an edge with a capacity of 0 exists.

Cut: A cut of a flow is the partition of vertices in a disjoint set (S, T) such that $S \cup T = V$, $s \in S$ and $t \in T$.

Cut Capacity: The cut capacity is the total maximum amount of flow that could possibly cross from the left side of the cut (the side with the source) to the right side (the side with the sink).

$$c(S, T) = \sum_{u \in S, v \in T} c(u, v)$$

NOTE: Reverse cuts do NOT count! For instance, an edge from cut T to S does NOT count!

Flow of Cut: The flow of a cut is how much flow is actually crossing from the source side (S) to the sink side (T) based on the current flow assignment. Formally, for a cut (S, T) , we define

$$f(S, T) = \sum_{u \in S, v \in T} f(u, v) - \sum_{u \in T, v \in S} f(u, v)$$

Max-flow min-cut theorem: The maximum value of an $s \rightarrow t$ flow is equal to the minimum capacity over all $s \rightarrow t$ cuts.

Problem: We want to maximize the flow from the source to the sink, or maximize $(|f|)$

4.1.3 More Definitions (Ford Fulkerson)

Residual Capacity of edge $c_f(u, v)$: It is the amount of additional flow that can pass through edge (u, v) ,

$$c_f(u, v) = c(u, v) - f(u, v)$$

Residual Network: A graph (network) $G_f = (V, E_f)$ where,

$$E_f = \{(u, v) \in V^2 : c_f(u, v) > 0\}$$

Augmented Path: An augmenting path is a path from the source to the sink where you can push more flow. Augmenting Path (AP): It is a simple path $s \xrightarrow{AP} t$ in G_f such that the residual capacity of AP:

$$c_f(p) = \min\{C_f(u, v) : (u, v) \in p\}$$

4.2 Ford Fulkerson & Edmond Karp

4.2.1 Ford Fulkerson

The algorithm repeatedly looks for a path from the source to the sink that can carry more flow. Every time you find such a path (an augmenting path), you push as much extra flow as possible through it. You keep doing this until there are no more paths that can carry extra flow. At that point, the flow you have achieved is the maximum flow.

The Pseudocode is presented below,

Algorithm 14 Ford-Fulkerson

```
1: procedure FORD-FULKERSON( $G, s, t$ )
2:   for each edge  $(u, v) \in G.E$  do
3:      $f(u, v) \leftarrow 0$ 
4:   end for
5:   while there exists a path  $p$  from  $s$  to  $t$  in the residual network  $G_f$  do
6:      $c_f(p) \leftarrow \min\{c_f(u, v) \mid (u, v) \text{ is in } p\}$ 
7:     for each edge  $(u, v)$  in  $p$  do
8:       if  $(u, v) \in E$  then
9:          $f(u, v) \leftarrow f(u, v) + c_f(p)$ 
10:      else
11:         $f(v, u) \leftarrow f(v, u) - c_f(p)$ 
12:      end if
13:    end for
14:  end while
15: end procedure
```

An example is shown below:

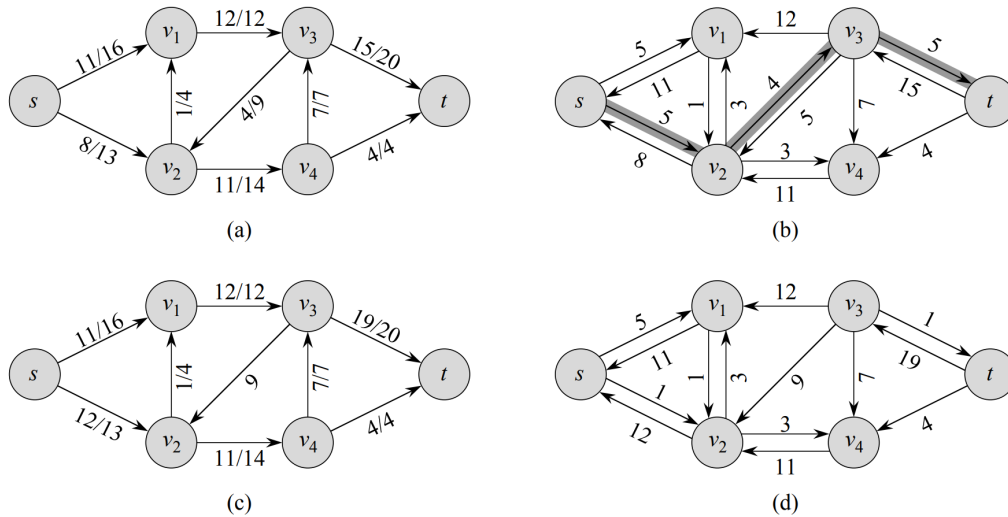


Figure 26.4 (a) The flow network G and flow f of Figure 26.1(b). (b) The residual network G_f with augmenting path p shaded; its residual capacity is $c_f(p) = c_f(v_2, v_3) = 4$. Edges with residual capacity equal to 0, such as (v_1, v_3) , are not shown, a convention we follow in the remainder of this section. (c) The flow in G that results from augmenting along path p by its residual capacity 4. Edges carrying no flow, such as (v_3, v_2) , are labeled only by their capacity, another convention we follow throughout. (d) The residual network induced by the flow in (c).

Figure 4.2: Ford–Fulkerson Example

Proof of correctness:

Max Flow - Min Cut Theorem: For all flow f on graph G , the following statements are equivalent:

1. f is a max-flow for G
2. G_f has no augmenting path
3. $|f| = c(S, T)$ For some cut

Runtime: $\mathcal{O}(|f_{\max}| \cdot E)$

4.2.2 Edmond Karp Algorithm

The Ford Fulkerson algorithm does not specify which path in the residual graph to augment; the poor worst-case behavior of the algorithm can be blamed on poor choices for the augmenting path. The Edmond-Karp algorithm resolves this issue by finding the augmenting path p with a breadth-first search.

This improves the runtime and completes in $\mathcal{O}(VE^2)$.

4.3 Maximum Bipartite Matching

Matching: A matching is a selection of edges from the graph such that no vertex is used more than once. Formally, the set of edges to form a mapping M is a subset $M \subseteq E$ in G such that $\forall v \in V$, at most one edge from M is incident on V .

Maximal Matching: you cannot add any more edges without breaking the matching rules.

Maximum Matching: A matching M such that for any matching M' , we have $|M| \geq |M'|$. A maximum matching is maximal.

To solve the Maximum Bipartite Matching problem, define a pseudo-source and pseudo-sink that connects to each of the vertices in each part of the graph that we are trying to match.

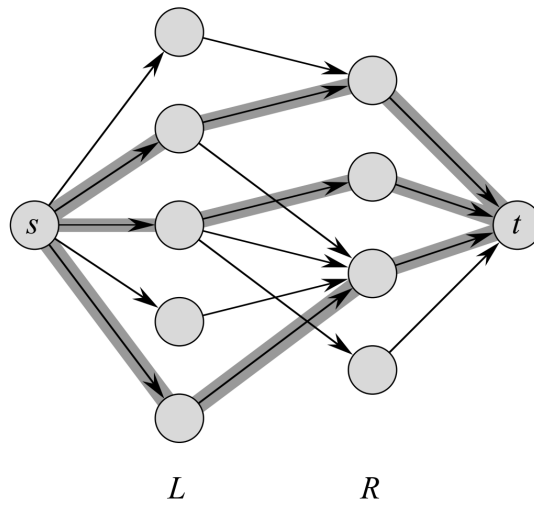


Figure 4.3: Maximum Bipartite Matching

Assign an capacity of ∞ to each edge from the pseudo-source and pseudo-sink to each of the nodes in the source and sink half respectively. For each of the original edges, we assign an capacity of 1. If we solve the maximum flow problem, the max-flow represents the size of a maximum matching in the bipartite graph. We can thus use Edmond Karp to find the matching, which solves the problem.

CHAPTER 5

Amortized Analysis

Amortized analysis: A technique used to average the time required to perform a sequence of data-structure operations over all operations performed.

Using amortized analysis, we can show that the average cost of an operation is small when averaged over a sequence of operations, even though a single operation may be expensive.

Amortized analysis is different from average-case analysis because probability is not involved. Instead, amortized analysis guarantees that the average performance of each operation, over the worst-case sequence of operations, is bounded.

5.1 Aggregate Method

Using the aggregate method, we compute the total cost of a sequence of operations and then divide by the number of operations.

$$\text{Amortized Cost} = \frac{\text{Total cost of } k \text{ operations}}{k}$$

Amortized Bounds: To establish amortized bounds, we assign an amortized cost to each operation such that the total amortized cost upper bounds the total actual cost.

$$\sum \text{Amortized Costs} \geq \sum \text{Actual Costs}$$

This ensures that the amortized cost of each operation reflects the worst-case performance of the algorithm when considered over a sequence of operations.

5.1.1 Example: Stack Operations

We know that the cost of push and pop operations on a stack is $\mathcal{O}(1)$. Therefore, the cost of pushing n items onto a stack is $\mathcal{O}(n)$.

Now, define a `MultiPop` operation, which pops the top k items from a stack. Consider a sequence of n `Push`, `Pop`, and `MultiPop` operations. The worst-case time for a single `MultiPop` is $\mathcal{O}(n)$. Thus, the worst-case time for any operation is also $\mathcal{O}(n)$.

Using a naive approach, for n operations, the total runtime could be

$$\mathcal{O}(n) \cdot n = \mathcal{O}(n^2)$$

However, using the aggregate method, we observe that each element can only be popped (or removed via `MultiPop`) at most once, which is the same number of times it was pushed. Therefore, the aggregate cost of all `Push`, `Pop`, and `MultiPop` operations is $\mathcal{O}(n)$ and the amortized cost per operation is $\mathcal{O}(1)$

5.1.2 Example: Binary Counter

Consider a binary counter that increments n times.

A naive approach flips every bit on each increment. Hence, the worst-case total cost is

$$\mathcal{O}(n \cdot n) = \mathcal{O}(n^2)$$

However, we can analyze more carefully:

- The least significant bit is incremented every time.
- The second bit is incremented every 2nd increment.
- The third bit is incremented every 4th increment, and so on.

Thus, for n increments, the total number of bit flips is at most

$$n + \frac{n}{2} + \frac{n}{4} + \cdots + 1 \leq 2n = \mathcal{O}(n)$$

Therefore, the amortized cost per increment is $\mathcal{O}(1)$ which is much better than the $\mathcal{O}(n)$ per increment suggested by the naive approach.

5.2 Accounting Method

The accounting method can be thought of as a bank account:

- Each operation can deposit or withdraw credit from the account.
- The bank balance cannot go negative.
- This allows "expensive" operations to be paid for using previously stored credit.

$$\text{Amortized Cost} = \text{Actual Cost} + \text{Deposits} - \text{Withdrawals}$$

Credit Invariant: A credit invariant is a rule describing where credits are stored, and how many credits are stored, such that every expensive operation can be fully paid for by previously stored credits.

A good invariant ensures every operation is paid for by its amortized cost + possibly spending or saving credits, and the total credits never become negative.

5.2.1 Example: Stack

Starting with an empty stack, we can charge:

- 2 coins for a push operation
- 1 coin for a pop operation

The credit invariant is: Every element currently stored in the stack must have 1 unit of saved credit.

- For the push, 1 coin pays for the actual operation, and 1 coin is stored as credit.
- For n push operations, we deposit n coins and pay n coins for the actual pushes.
- When we pop, we use the stored credit to pay for the operation. Popping n items costs n coins in credit.

Thus, the total amortized cost is

$$\text{Amortized Cost} = n + n - n = \mathcal{O}(n)$$

5.2.2 Example: Binary Counter

For a binary counter, consider flipping a bit from 0 to 1:

- Charge 2 coins per flip:
 - 1 coin pays for the actual operation
 - 1 coin is stored as credit
- The stored credit pays for subsequent flips from 1 to 0.

The credit invariant is: Every bit that is 1 must have 1 unit of stored credit.

Hence, for n increments of the counter, the total amortized cost is

$$n + n - n = \mathcal{O}(n)$$

5.3 Splay Trees

A **Splay Tree** is a self-balancing binary search tree (BST) with no explicit balancing information. The goal of a splay tree is to bring the most recently accessed node to the top of the tree. To achieve this, we use **rotations**.

5.3.1 Definitions of Rotations

- **Zig/Zag:** Single rotation.

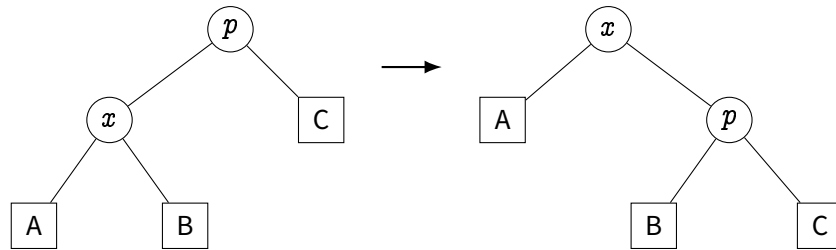


Figure 5.1: Zig Operation

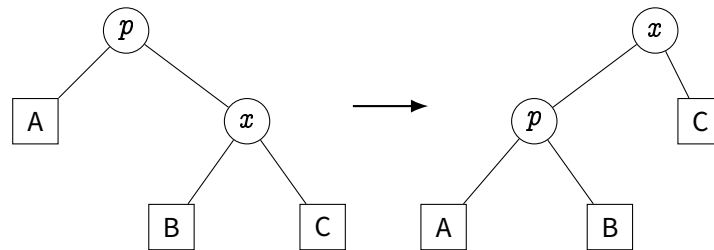


Figure 5.2: Zag Operation

- **Zig-Zig/Zag-Zag:** Double rotation in the same direction (e.g., left-left or right-right).

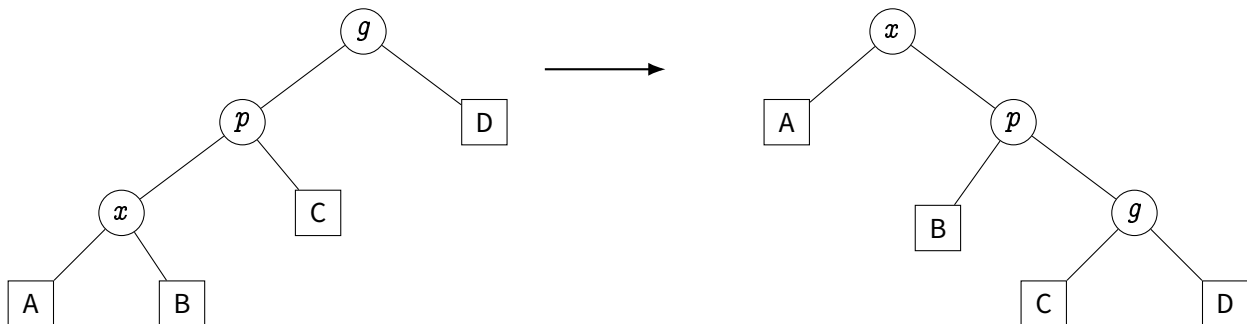


Figure 5.3: Zig-Zig Operation

- **Zig-Zag/Zag-Zig:** Double rotation in opposite directions (e.g., left-right or right-left).

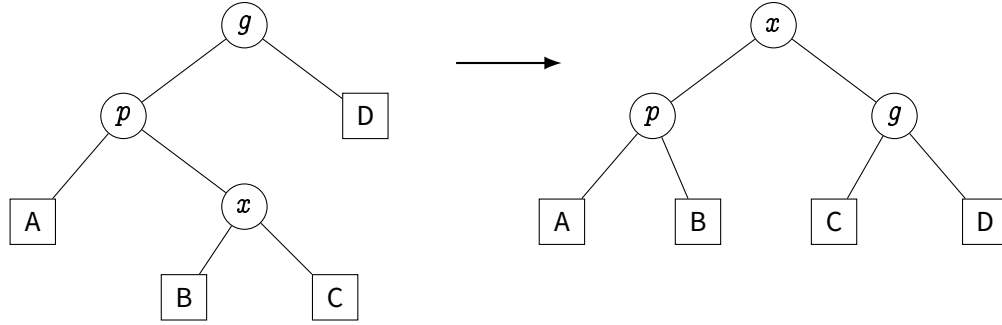


Figure 5.4: Zig-Zag Operation

Splay Step: The key operation in a splay tree is the **splay step**, which moves a node x to the root.

Case 0: If x 's parent is the root, perform a single rotation (zig or zag) to make x the root.

Case 1: If x and its parent are both left children or both right children, perform a zig-zig or zag-zag rotation to make x the root.

Case 2: If x is a left child and its parent is a right child (or vice versa), perform a zig-zag or zag-zig rotation to make x the root.

5.3.2 Operations

Insert(x): Insert the new node as in a standard BST, then splay the node to move it to the root.

Delete(x): Like in a BST, find x , replace the root if necessary, then splay the parent of x , i.e., $\text{splay}(\text{parent}(x))$.

Split(x): Search for x , splay x to the root, then the left and right children of the root are the split halves of the tree.

Join(x): Find x , then attach two subtrees H and H' to the left and right of x , respectively.

5.3.3 Cost of Splay Operation

First, we define some terms:

- **Weight:** $WT(x)$ — the number of nodes in the subtree rooted at x , including x itself.
- **Rank:** $\text{rank}(x) = \log(WT(x))$

Lemma 1: Every zig, zag, zig-zig, and zig-zag operation has cost

$$d \leq 3(\text{newrank}(x) - \text{oldrank}(x))$$

except if $\text{parent}(x)$ is the root, in which case

$$d \leq 3(\text{newrank}(x) - \text{oldrank}(x)) + 1$$

Using this lemma, the amortized cost of splaying a node x over k steps (whether zig, zag, zig-zig, or zig-zag) is

$$\text{Amortized Cost} = \sum_{i=1}^k 3(\text{newrank}(x_i) - \text{oldrank}(x_i)) \quad \underbrace{+1}_{\text{cost of at least one ZAG}}$$

Note:

- oldrank = rank before the rotation
- newrank = rank after the rotation

Hence, the sum evaluates to

$$\begin{aligned} \sum_{i=0}^k (\text{newrank}(x) - \text{oldrank}(x)) &= (\text{rank}(1) - \text{rank}(0)) \\ &\quad + (\text{rank}(2) - \text{rank}(1)) \\ &\quad + \dots \\ &\quad + (\text{rank}(k) - \text{rank}(k-1)) \\ &= \text{rank}(k) - \text{rank}(0) \end{aligned}$$

Thus, the amortized cost satisfies

$$\text{Amortized Cost} \leq 3(\text{rank}(k) - \text{rank}(0)) + 1 = 3(\log n - 0) + 1 = O(\log n) + 1$$

Hence, we can conclude that a splay operation has amortized time complexity $O(\log n)$

CHAPTER 6

Dynamic Programming

6.1 Matrix-Chain Multiplication

Problem

Given matrices $A_1, A_2, \dots, A_n$ where A_i has dimension $p_{i-1} \times p_i$, compute the parenthesization of the product $A_1 A_2 \cdots A_n$ that minimizes the total number of scalar multiplications.

Deriving the Recurrence

Consider any optimal parenthesization of $A_i \cdots A_j$ (with $i < j$). The last multiplication splits the chain between some k with $i \leq k < j$:

$$(A_i \cdots A_k) \cdot (A_{k+1} \cdots A_j)$$

If we know the minimum costs to compute the left and right subproducts, the cost to multiply the two resulting matrices is

$$\underbrace{m[i, k]}_{\text{cost left}} + \underbrace{m[k+1, j]}_{\text{cost right}} + \underbrace{p_{i-1} p_k p_j}_{\text{cost to multiply resulting matrices}}.$$

Since the optimal split k is the one minimizing that total, we obtain the recurrence

$$m[i, j] = \begin{cases} 0 & \text{if } i = j, \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1} p_k p_j\} & \text{if } i < j. \end{cases}$$

DP data structure and algorithm idea

We store:

- $m[i, j]$: minimum cost (number of scalar multiplications) to compute $A_i \cdots A_j$,
- $s[i, j]$: index k at which we split the optimal multiplication for $A_i \cdots A_j$ (used to reconstruct the parenthesization).

For each length ℓ :

1. Iterate i from 1 to $n - \ell + 1$ and set $j = i + \ell - 1$.
2. Compute $m[i, j]$ using the recurrence above.

This bottom-up approach ensures that when we compute a subchain of length ℓ , all shorter subchains (which appear on the right-hand side of the recurrence) have already been computed.

- Time complexity: there are $\Theta(n^2)$ entries and computing each $m[i, j]$ does up to $O(n)$ work $\Rightarrow O(n^3)$ time.
- Space: $O(n^2)$ for m and s .

Example

Given 6 matrices with dimensions:

$$\begin{cases} A_1 = 30 \times 35 \\ A_2 = 35 \times 15 \\ A_3 = 15 \times 5 \\ A_4 = 5 \times 10 \\ A_5 = 10 \times 20 \\ A_6 = 20 \times 25 \end{cases}$$

In this example, we can observe that,

$$p = \{30, 35, 15, 5, 10, 20, 25\}$$

We compute $m[i, j]$ and $s[i, j]$ by increasing chain length.

The sequence is as follows:

1. We set $m[i, i]$ to be 0. We set $s[i, i] = i$
2. Going to row $j = 2$, we compute $m[1, 2]$. By definition, we have,

$$m[1, 2] = \underbrace{0}_{m[1,1]} + \underbrace{0}_{m[2,2]} + \underbrace{30 \times 35 \times 15}_{p_0 p_1 p_2} = 15,750$$

Set $s[1, 2] = k = 1$

3. Going to the next column, we compute $m[2, 3]$. By definition, we have,

$$m[2, 3] = \underbrace{0}_{m[2,2]} + \underbrace{0}_{m[2,3]} + \underbrace{35 \times 15 \times 5}_{p_1 p_2 p_3} = 2,625$$

Set $s[2, 3] = k = 2$

4. We repeat until we have the complete $j = 2$ row filled. Moving to $j = 3$ row, there are two possibilities:

$$m[1, 3] = \begin{cases} \underbrace{0}_{m[1,1]} + \underbrace{2,625}_{m[2,3]} + \underbrace{30 \times 35 \times 5}_{p_0 p_1 p_3} = 7,875 & k = 1 \text{ (left branch)} \\ \underbrace{15,750}_{m[1,2]} + \underbrace{0}_{m[3,3]} + \underbrace{30 \times 15 \times 5}_{p_0 p_2 p_3} = 18,000 & k = 2 \text{ (right branch)} \end{cases}$$

By observation, 7,875 is smaller, so we take $m[1, 3] = 7,875$ and $s[1, 3] = k = 1$.

5. Continue until the $j = 3$ row is filled. Moving to $j = 4$ row, there are three possibilities:

$$m[1, 4] = \begin{cases} \underbrace{0}_{m[1,1]} + \underbrace{4,375}_{m[2,4]} + \underbrace{30 \times 35 \times 10}_{p_0 p_1 p_4} = 14,875 & k = 1 \text{ (left branch)} \\ \underbrace{15,750}_{m[1,2]} + \underbrace{750}_{m[3,4]} + \underbrace{30 \times 15 \times 10}_{p_0 p_2 p_4} = 21,000 & k = 2 \text{ (center branch)} \\ \underbrace{7,875}_{m[1,3]} + \underbrace{0}_{m[4,4]} + \underbrace{30 \times 5 \times 10}_{p_0 p_3 p_4} = 9,375 & k = 3 \text{ (right branch)} \end{cases}$$

By observation, 9,375 is smaller, so we take $m[1, 4] = 9,375$ and $s[1, 4] = k = 3$.

6. Repeat until table is filled.

The fully computed tables (rows indexed by i , columns by j) are:

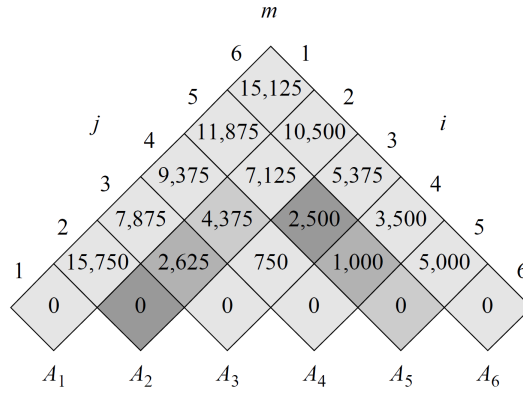


Figure 6.1: DP Data Structure for m

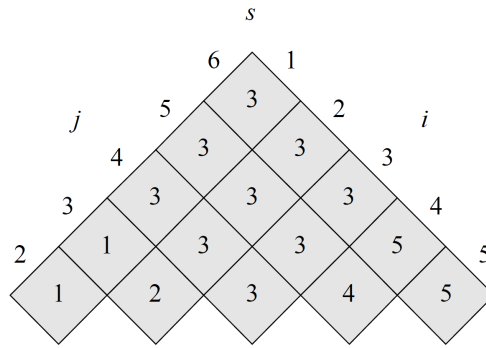


Figure 6.2: DP Data Structure for s

So the optimal parenthesization is

$$((A_1(A_2A_3))((A_4A_5)A_6)),$$

which attains the minimal cost 15125 scalar multiplications.

6.2 Longest Common Subsequence (LCS)

Problem

Given two string,

$$X = x_1x_2 \dots x_m$$

$$Y = y_1y_2 \dots y_n$$

Find subsequence (not necessarily consecutive but in the original order) which is common to both strings.

Deriving the Recurrence

To derive this recurrence, we first start with a proof of a theorem:

Theorem: Let $X = \langle x_1, x_2, \dots, x_m \rangle$ and $Y = \langle y_1, y_2, \dots, y_n \rangle$ be sequences, and let $Z = \langle z_1, z_2, \dots, z_k \rangle$ be any longest common subsequence (LCS) of X and Y . Then:

1. If $x_m = y_n$, then $z_k = x_m = y_n$, and Z_{k-1} is an LCS of X_{m-1} and Y_{n-1} .
2. If $x_m \neq y_n$ and $z_k \neq x_m$, then Z is an LCS of X_{m-1} and Y .
3. If $x_m \neq y_n$ and $z_k \neq y_n$, then Z is an LCS of X and Y_{n-1} .

Proof. We consider the relationship between the last characters of X and Y .

Case 1: $x_m = y_n$.

Since the last characters of X and Y are the same, any common subsequence of X_{m-1} and Y_{n-1} can be extended by this character to form a common subsequence of X and Y . If $z_k \neq x_m$, then we could append $x_m = y_n$ to Z and obtain a common subsequence of X and Y of length $k + 1$, contradicting the assumption that Z is a longest common subsequence. Therefore, $z_k = x_m = y_n$.

Now, consider the prefix $Z_{k-1} = \langle z_1, \dots, z_{k-1} \rangle$. It is a common subsequence of X_{m-1} and Y_{n-1} . If there existed a longer common subsequence W of X_{m-1} and Y_{n-1} , then appending $x_m = y_n$ to W would yield a common subsequence of X and Y of length greater than k , which is impossible. Hence, Z_{k-1} is an LCS of X_{m-1} and Y_{n-1} .

Case 2: $x_m \neq y_n$ and $z_k \neq x_m$.

Since the last element of X does not match the last element of Y and z_k is not equal to x_m , the character x_m does not contribute to the LCS. Therefore, Z must be a common subsequence of X_{m-1} and Y . If there existed a longer common subsequence W of X_{m-1} and Y , then W would also be a common subsequence of X and Y , contradicting the maximality of Z . Thus, Z is an LCS of X_{m-1} and Y .

Case 3: $x_m \neq y_n$ and $z_k \neq y_n$.

This case is symmetric to Case 2. Since y_n does not appear in the LCS, Z is a common subsequence of X and Y_{n-1} . If there were a longer common subsequence W of X and Y_{n-1} , then W would also be a common subsequence of X and Y , again contradicting the maximality of Z . Therefore, Z is an LCS of X and Y_{n-1} .

In all cases, the LCS of X and Y can be expressed in terms of the LCS of their prefixes, demonstrating the optimal substructure property. ■

From this theorem, we can define,

$$c[i, j] = \text{length of LCS}(X[1 \dots i], Y[1 \dots j]).$$

Observe, we have three cases:

1. If $i = 0$ or $j = 0$, then one of the strings is empty. You cannot have a common sequence if one of the sequence is empty, so $c[i, j] = 0$.
2. If $x_i = x_j$, then from part 1 of the theorem we know that this is a common character, so we find the common sequence of characters before it, plus that additional character. So, $c[i - 1, j - 1] + 1$.
3. If $x_i \neq x_j$, then from part 2 and 3 of the theorem, we know that the longest common sequence is either between X_{i-1} and Y or X and Y_{j-1} , whichever is longer. So, $c[i, j] = \max(c[i - 1, j], c[i, j - 1])$

And so, we have the recurrence,

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i - 1, j - 1] + 1 & \text{if } x_i = x_j \\ \max(c[i - 1, j], c[i, j - 1]) & \text{if } x_i \neq y_j. \end{cases}$$

Example

Take

$$X = \text{AMPUTATION} \quad (m = 10) \text{ and } Y = \text{SPANKING} \quad (n = 8)$$

To solve this, we will use an dynamic programming array of sized $m \times n$.

1. By the base case of the recurrence, the first row and column of this array are all 0s.
2. For the second row,
 - (a) If the character in the column is equal to the character in the row, by the second condition of the recurrence, we set $c[i, j]$ to be the value of the value in the upper diagonal +1.
 - (b) If not, we compare the value above and left to it, and take the value greater, whatever that may be.
3. Repeat until the table is filled.

4. The LCS is the sequence of characters where

The computed table $c[i, j]$ (rows indexed by i , columns by j) is:

		A	M	P	U	T	A	T	I	O	N
	0	0	0	0	0	0	0	0	0	0	0
S	0	0	0	0	0	0	0	0	0	0	0
P	0	0	0	1	1	1	1	1	1	1	1
A	0	1	1	1	1	1	2	2	2	2	2
N	0	1	1	1	1	1	2	2	2	2	3
K	0	1	1	1	1	1	2	2	2	2	3
I	0	1	1	1	1	1	2	2	3	3	3
N	0	1	1	1	1	1	2	2	3	3	4
G	0	1	1	1	1	1	2	2	3	3	4

Table 6.1: LCS Dynamic Programming Array

And so, the longest common sequence is of length 4, which is PAIN.

Complexity: computing the table takes $O(mn)$ time and $O(mn)$ space. If you only need the length, you can reduce the space to $O(\min(m, n))$.

CHAPTER 7

Greedy Algorithms

7.1 Activity-Selection

Problem Statement

Given activities $a_1, \dots, a_n$ with start times s_i and finish times f_i , select a maximum-size subset of mutually compatible activities (no overlap).

Recurrence

Assume activities $a_1, \dots, a_n$ are sorted by nondecreasing finish time. Introduce two dummy activities a_0 and a_{n+1} with $f_0 = 0$ and $s_{n+1} = +\infty$ to simplify boundary conditions.

For indices $i < j$ define

$$S_{ij} = \{a_k \mid s_k \geq f_i \text{ and } f_k \leq s_j\},$$

the set of activities that start no earlier than f_i and finish no later than s_j .

Let $c[i, j]$ denote the size (number of activities) of an optimal selection from S_{ij} .

If $S_{ij} = \emptyset$, then there are no compatible activities in the interval and $c[i, j] = 0$.

If $S_{ij} \neq \emptyset$, consider any optimal solution A^* for S_{ij} . If A^* contains some activity, choose any $a_k \in A^*$. Because activities in A^* do not overlap, the activities of A^* split into three disjoint parts:

$$\underbrace{A^* \cap S_{i,k}}_{\text{left}} \cup \{a_k\} \cup \underbrace{A^* \cap S_{k,j}}_{\text{right}}.$$

The left part must be an optimal solution for $S_{i,k}$ and the right part must be optimal for $S_{k,j}$, otherwise we could replace a non-optimal part by an optimal one and increase $|A^*|$, contradicting optimality. Hence, if the optimal solution uses activity a_k we have

$$|A^*| = c[i, k] + 1 + c[k, j].$$

Since the optimal solution for S_{ij} either is empty or uses some $a_k \in S_{ij}$, we take a maximum over all choices of k :

$$c[i, j] = \begin{cases} 0, & \text{if } S_{ij} = \emptyset, \\ \max_{a_k \in S_{ij}} (c[i, k] + 1 + c[k, j]), & \text{otherwise.} \end{cases}$$

Greedy idea

Sort activities by nondecreasing finish times f_i . Repeatedly pick the earliest-finishing activity that starts after (or at) the finish time of the last selected activity.

Example

The activities (already ordered by finish times) are:

Activity	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}	a_{11}
Start s_i	1	3	0	5	3	5	6	8	8	2	12
Finish f_i	4	5	6	7	8	9	10	11	12	13	14

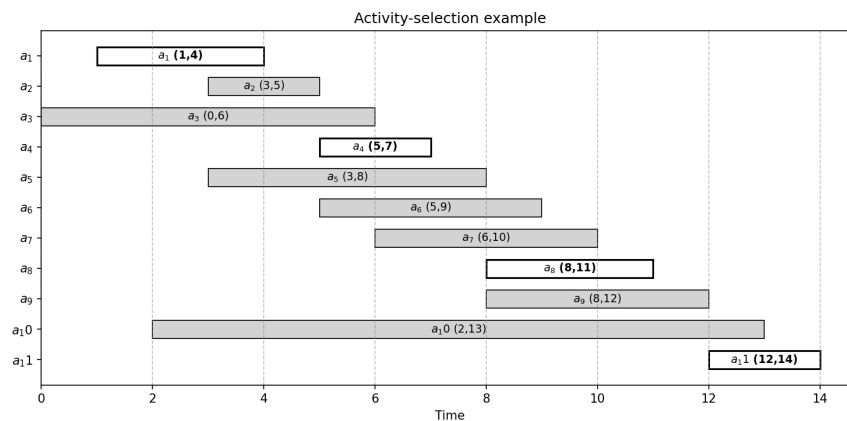


Figure 7.1: Activities Plot

Run of the greedy algorithm

- Pick a_1 (finishes at 4).
- Next activity with start ≥ 4 is a_4 (starts 5, finishes 7) $\Rightarrow$ pick a_4 .
- Next with start ≥ 7 is a_8 (starts 8, finishes 11) $\Rightarrow$ pick a_8 .
- Next with start ≥ 11 is a_{11} (starts 12, finishes 14) $\Rightarrow$ pick a_{11} .

So the greedy algorithm returns the set

$$A = \{a_1, a_4, a_8, a_{11}\}.$$

7.2 The Knapsack Problems

7.2.1 The Fractional Knapsack Problem

Problem Statement

A thief robbing a store finds n items. The i th item is worth v_i dollars and weighs w_i pounds, where v_i and w_i are integers. The thief wants to take as valuable a load as possible, and he can carry fractions of items. Which items, and of what proportions, should he take?

Solution (Greedy)

In the fractional variant, we are allowed to take any fraction of an item. That is,

$$0 \leq x_i \leq 1, \quad i = 1, 2, \dots, n.$$

The objective and constraint remain the same:

$$\begin{aligned} & \text{maximize } \sum_{i=1}^n v_i x_i, \\ & \text{subject to } \sum_{i=1}^n w_i x_i \leq W. \end{aligned}$$

The fractional knapsack problem can be solved optimally using a greedy algorithm:

Sort items by decreasing value-to-weight ratio v_i/w_i , then take as much as possible of each item in that order until the knapsack is full.

This algorithm runs in $O(n \log n)$ time, as it is just sorting the n items.

7.2.2 The 0–1 Knapsack Problem

Problem Statement

A thief robbing a store finds n items. The i th item is worth v_i dollars and weighs w_i pounds, where v_i and w_i are integers. The thief wants to take as valuable a load as possible, but he can carry at most W pounds in his knapsack, for some integer W . Which items should he take?

Solution (Dynamic Programming)

We are given a set of n items, where each item i has:

$$\text{value } v_i > 0, \quad \text{and weight } w_i > 0.$$

We also have a knapsack with a maximum capacity $W > 0$.

The goal is to select a subset of items to maximize the total value without exceeding the knapsack's capacity. Formally, we define indicator variables

$$x_i = \begin{cases} 1 & \text{if item } i \text{ is included in the knapsack,} \\ 0 & \text{otherwise.} \end{cases}$$

The problem can be formulated as:

$$\begin{aligned} & \text{maximize } \sum_{i=1}^n v_i x_i, \\ & \text{subject to } \sum_{i=1}^n w_i x_i \leq W, \\ & \quad x_i \in \{0, 1\}, i = 1, 2, \dots, n. \end{aligned}$$

This is the 0–1 knapsack problem, since each item must either be taken in whole or not taken at all. It exhibits optimal substructure and overlapping subproblems, and can be solved using dynamic programming.

Let $c[i, w]$ denote the maximum total value achievable using the first i items with a total capacity limit w . The recurrence relation is defined as:

$$c[i, w] = \begin{cases} 0, & \text{if } i = 0 \text{ or } w = 0, \\ c[i - 1, w], & \text{if } w_i > w, \\ \max(c[i - 1, w], v_i + c[i - 1, w - w_i]), & \text{if } w_i \leq w. \end{cases}$$

This algorithm can run in $O(nW)$ time.

Comparison. The key difference between the two problems is that the fractional knapsack allows continuous fractions of items and is solvable greedily, whereas the 0–1 knapsack is combinatorial and requires dynamic programming or other exact or approximate methods.

7.3 Huffman's Encoding

Problem Statement

Given a set of characters $C = \{c_1, c_2, \dots, c_n\}$ with frequencies $f(c_1), f(c_2), \dots, f(c_n)$, we wish to construct a binary prefix code that minimizes the average codeword length.

If $d(c)$ is the length of the codeword assigned to symbol c , the objective is to minimize

$$B(T) = \sum_{c \in C} f(c) d(c).$$

You want to minimize $B(T)$, i.e., # of bits to encode the file.

Huffman's Algorithm

1. Initialize a forest of n single-node trees, one for each character $c \in C$, with weight $f(c)$.
2. Repeat until there is only one tree left:
 - (a) Select two trees T_1 and T_2 with the smallest weights.
 - (b) Create a new node as their parent with weight equal to $w(T_1) + w(T_2)$.
 - (c) Make T_1 and T_2 the left and right subtrees of this node.
3. The remaining single tree is the Huffman tree. Assign binary digits 0 and 1 to left and right branches, respectively.

The codeword for each character is the sequence of bits encountered on the path from the root to its leaf.

Example

Given the following frequency table,

Character	a	b	c	d	e	f
Frequency	45	13	12	16	9	5

Table 7.1: Character frequencies for the Huffman coding example.

We can create the following Huffman tree by using the Huffman algorithm.

Proof of Optimality

Greedy Principle: Every optimal tree can have the two lowest frequency keys being siblings that differ in only 1-bit.

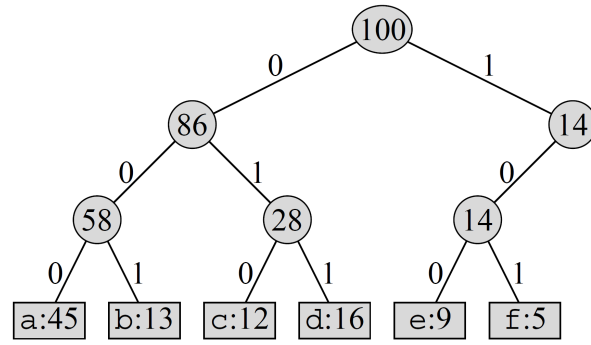


Figure 7.2: Huffman Tree

Proof. Assume for the sake of contradiction that the keys x & y are the lowest frequency but they do not differ by a bit. We will replace x with some other character b and show that,

$$B(T) \geq B(T')$$

Let T be a tree where,

- x and y are two symbols that have the smallest frequency.
- a and b be two leaves that are siblings at maximum depth in T^* .

We want to show that there exists an optimal tree for C having x and y as siblings at maximum depth.

If we exchange the positions of x with a to create a tree T' ,

$$\begin{aligned} B(T) - B(T') &= [f(a)d(a) + f(x)d(x)] - [f(a)d(x) + f(x)d(a)] \\ &= f(a)[d(a) - d(x)] - f(x)[d(a) - d(x)] \\ &= [f(a) - f(x)][d(a) - d(x)] \end{aligned}$$

Since x has the lowest frequency, $f(a) \geq f(x)$. Since a is the lowest height, $d(a) \geq d(x)$

Thus, $B(T) - B(T') \geq 0 \Rightarrow B(T) \geq B(T')$. This implies that T' is optimal. But we already said T is optimal. A contradiction!

We can do the same with b and y to attain the same result.

Thus, since assuming x and y are not at the max depth creates a contradiction, we can conclude that x and y have the optimal subtree at the maximum depth. ■

Optimal Substructure: Let C be an alphabet with a frequency $f[c]$ defined for each character $c \in C$. Let x and y be the two characters in C with the smallest frequencies. Define a new alphabet

$$C' = (C - \{x, y\}) \cup \{z\},$$

where z is a new character with frequency

$$f[z] = f[x] + f[y].$$

Let T' be an optimal prefix code tree for the alphabet C' .

Then the tree T , obtained from T' by adding x and y for z where having x and y as children, is an optimal prefix code tree for the original alphabet C .

Proof. To setup the proof, we first show how to express the cost $B(T)$ of tree T in terms of the cost of $B(T')$ of tree T' .

We know that by definition, $d(x) = d(y) = d(z) + 1$. Thus,

$$\begin{aligned} f(x)d(x) + f(y)d(y) &= f(x)(d(z) + 1) + f(y)(d(z) + 1) \\ &= (f(x) + f(y))(d(z) + 1) \\ &= f(z)d(z) + (f(x) + f(y)) \\ &= B(T') + (f(x) + f(y)) \end{aligned}$$

Now, to prove that T is optimal, I will prove this using contradiction.

Suppose for the sake of contradiction that T does not represent an optimal prefix code for C . Then, there exists an optimal tree T'' such that,

$$B(T'') < B(T)$$

Let T''' be the tree T'' with the common parent of x and y replaced by a leaf z . Then, from the above,

$$\begin{aligned} B(T'') &= B(T''') + (f(x) + f(y)) \\ B(T''') &= B(T'') - (f(x) + f(y)) \\ &< B(T) - (f(x) + f(y)) \\ &= B(T') \end{aligned}$$

Thus, $B(T''') < B(T')$, meaning we have found a tree for C' with smaller cost than T' . But we assumed that T' is optimal for C' . A contradiction!

Therefore our assumption that T is not optimal must be false. Hence T is an optimal prefix code tree for C . ■

CHAPTER 8

Algorithmic Complexity

8.1 Definitions

Problems: A **computational problem** is a general question that we want an algorithm to answer. In this context, we focus on **decision problems**, whose answer is either **YES** or **NO**.

More formally, **decision problem** is defined as a language

$$L \subseteq \Sigma^*,$$

where Σ is a finite alphabet and Σ^* is the set of all finite strings over Σ . Each string $x \in \Sigma^*$ represents a possible input to the problem. If $x \in L$, the correct output is **YES**; otherwise, the correct output is **NO**.

Problem Instances: A **problem instance** is one specific input to a computational problem.

More formally, a **problem instance** is a specific input $x \in \Sigma^*$ to a computational problem L . The instance is a **YES-instance** if $x \in L$, and a **NO-instance** if $x \notin L$.

Satisfiability: There exists at least one assignment of truth values (true/false or 1/0) to the inputs such that the output is true (1).

8.2 Problem Classes

The following defines the different classes of problems.

8.2.1 Class P

Class P is the set of all problems that can be solved in polynomial time. Mathematically,

$$P = \{L = \{0, 1\}^* \text{ there exists algorithm } A \text{ that decides } L \text{ in polynomial time}\}$$

Alternatively, class P is any problem that can be solved in $\mathcal{O}(n^k)$ for any $k \in \mathbb{N}$. This leads to the following theorem.

Theorem: If a problem can be solved by an algorithm that runs in polynomial time, then it belongs to the complexity class P .

Proof. By definition, a language L that is accepted by a machine A in polynomial time means for every input string of length n , machine A halts within cn^k steps.

We construct a new machine A' that simulates A on x for at most cn^k steps. If A halts and accepts within this time, then A' accepts. Otherwise, if A does not halt within cn^k steps, A' rejects.

Since A' performs at most cn^k steps, it runs in polynomial time. Moreover, A' decides the same language as A . Therefore, the language belongs to class P . ■

8.2.2 Class NP

Verification: We say that a verifier algorithm A **verifies** a problem if given an instance $x \in L$ of the problem, there exists a certificate (or witness) $y \in L$ such that $A(x, y) = 1$. The language we verified is,

$$L = \{x \in \{0, 1\}^* : \exists y \in \{0, 1\}^* \text{ such that } A(x, y) = 1\}$$

The certificate y is the candidate solution that we are trying to verify. You can think of A as taking in a problem and a solution, and it returns 1 if that solution correctly solves the problem.

Class-NP: It is the class of problems that have a polynomial-time verification algorithm.

Formally, a language (problem) L belongs to complexity class NP if there exists a polynomial time algorithm A and a constant c such that L contains the string:

$$L = \{x \in \{0, 1\}^* : \exists \text{certificate } y, |y| = \mathcal{O}(|x|^c) \text{ such that } A(x, y) = 1\}$$

Theorem: $P \subseteq NP$

Proof. If $L \in P$, that means there exists an algorithm to decide (solve) L in polynomial time. Hence, we can just ignore the certificate and solve the problem in polynomial time. ■

Co-NP: If the answer to a problem in co-NP is **NO**, then there is a proof of this fact that can be checked in polynomial time. Essentially, this is the opposite of NP.

Theorem: Class P is closed under complement. That is,

$$L \in P \Rightarrow \bar{L} \in P$$

Proof. Assume $L \in P$. By definition of class P , there exists a deterministic Turing machine A that decides L in polynomial time.

We construct a new deterministic Turing machine A' as follows.

- On input x , machine A' runs machine A on x .
- If A accepts, then A' rejects.
- If A rejects, then A' accepts.

Since A always halts and runs in polynomial time, machine A' also always halts and runs in polynomial time. Moreover, A' accepts exactly those strings not accepted by A .

Therefore, A' decides the language $\bar{L}$ in polynomial time. Hence, $\bar{L} \in P$. ■

8.3 NP Completeness

Reducibility: Reducing problem A to another problem B means describing an algorithm to solve problem A under the assumption that an algorithm for problem B already exists.

We say that $A \leq_p B$ (“ A is P reducible to B ”) if and only if for all polynomial-time algorithms that gives an instance of A , we can transform it to some instance of B , solve for B , and then in polynomial time obtain a solution for A .

Formally, we say that language L is polynomially reducible to L_2 , denoted as,

$$L_1 \leq_p L_2$$

If and only if there exists a polynomial-time algorithm $f : L_1 \rightarrow L_2$. That is, if we can answer if $f(x) \in L_2$, we can answer if $x \in L_1$.

Theorem (Lemma 34.3): If $L_1 \leq_p L_2$ and $L_2 \in P$, then $L_1 \in P$.

Proof. Assume $L_1 \leq_p L_2$. Then, by definition, there exists a polynomial-time algorithm $f : L_1 \rightarrow L_2$.

Assume $L_2 \in P$. Then, there exists a machine $M : L_2 \rightarrow \{\mathbf{YES}, \mathbf{NO}\}$ that solves L_2 in polynomial time.

We construct a deterministic Turing machine M' that decides L_1 as follows.

- On input x , machine M' first computes $f(x)$ and then runs M on input $f(x)$.
- Machine M' accepts if and only if M accepts $f(x)$.

Since f is computable in polynomial time and M runs in polynomial time, their composition runs in polynomial time. Moreover, by the correctness of the reduction, M' accepts exactly those strings in L_1 .

Therefore, $L_1 \in P$. ■

NP-Completeness: We say that a language L is NP-complete (NPC) if and only if,

- $L \in NP$. In other words, L can be verified in polynomial time
- $\forall L' \in NP, L' \leq_P L$. In other words, every problem in NP can be reduced to an NP-complete problem. This is the **NP-Hardness** condition

The relationship between all of the complexity classes is shown below,

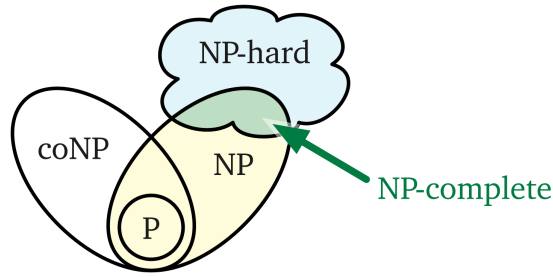


Figure 8.1: Complexity Diagram

Theorem (Theorem 34.4): There exists a language $L \in NPC$ such that if $L \in P$, then $P = NP$.

Proof. Suppose that $L \in P$ and also that $L \in NPC$. For any $L' \in NP$, we have $L' \leq_P L$ by property 2 of the definition of NP-completeness. Thus, since $L \in P$ and $L' \leq_P L$, we can deduce $L' \in P$, which proves the first statement of the theorem. ■

Theorem: $NP = co-NP \Leftrightarrow \exists L \in NPC$ such that $\bar{L} \in NPC$

Proof. ($\Rightarrow$) Assume $NP = co-NP$. Let $L \in NPC$. Since $L \in NP$, its complement $\bar{L}$ is in $co-NP$. By the assumption that $NP = co-NP$, it follows that $\bar{L} \in NP$.

To show that $\bar{L} \in NPC$, let $L' \in NP$ be an arbitrary language. Because L is NP-complete, we have $L' \leq_P L$. Complementing both sides gives $\bar{L}' \leq_P \bar{L}$. Since $NP = co-NP$, $\bar{L}' \in NP$, so every language in NP polynomial-time reduces to $\bar{L}$. Thus, $\bar{L}$ is NP-hard and belongs to NP , hence $\bar{L} \in NPC$.

($\Leftarrow$) Assume there exists a language $L \in NPC$ such that $\bar{L} \in NPC$. Let $L' \in NP$ be arbitrary. Since $\bar{L}$ is NP-complete, we have $L' \leq_P \bar{L}$. By closure of polynomial-time reductions under complement, this implies $\bar{L}' \leq_P L$.

Because $L \in NP$, it follows that $\bar{A} \in NP$, and hence $A \in co-NP$. Since $A \in NP$ was arbitrary, we conclude that $NP \subseteq co-NP$. The reverse containment always holds by symmetry, so $NP = co-NP$. ■

8.3.1 NP-Completeness Methodology

Proving directly that a language is NP-complete can be difficult. Instead, we use the following standard methodology.

Let L be a language. To prove that $L \in NPC$, it suffices to show:

1. **$L \in NP$:** There exists a polynomial-time verification algorithm for L (i.e., given an instance and a certificate, membership in L can be verified in polynomial time).

Intuition: Given a proposed solution, we can efficiently check that it is correct. In other words, solutions may be hard to find, but they are easy to verify.

2. **L is NP-hard:** Select a known NP-complete language $L' \in NPC$ and show that $L' \leq_P L$ by:

- (a) Describing a polynomial-time reduction f that maps each instance $x \in \Sigma^*$ of L' to an instance $f(x)$ such that

$$x \in L' \Leftrightarrow f(x) \in L.$$

Intuition: Any instance of the already-known hard problem L' can be efficiently transformed into an instance of L . This shows that solving L is at least as hard as solving L' .

- (b) Proving that the function f can be computed in polynomial time.

Intuition: The transformation itself must be efficient; otherwise, the reduction would not preserve computational difficulty.

This is sufficient to proving NP-completeness as if $L \in NPC$, this implies that all NP problems can be reduced to this NPC problem, which satisfies the second condition of NPC.

But, to use this methodology, you need a solution to an NPC problem, which now creates a circular logic.

Circuit Satisfiability Problem: Given a AND, OR, and NOT digital circuits with a single output, find an input vector that makes the output 1.

Lemma (Lemma 34.5): The Circuit Satisfiability Problem belongs to class NP.

Lemma (Lemma 34.6): The Circuit Satisfiability Problem belongs to class NP-Hard.

Cook-Levin Theorem (Theorem 34.7): The circuit-satisfiability problem is NP-complete.

Implication: We can polynomial reduce any NP problem to the Circuit Satisfiability Problem.

8.4 Examples

The following are examples of the NP-Completeness Methodology.

8.4.1 Formula Set Satisfiability

We formulate the (formula) satisfiability problem in terms of the language SAT as follows. An instance of SAT is a Boolean formula ϕ composed of:

1. n Boolean variables: $x_1, x_2, \dots, x_n$;
2. m Boolean connectives, where each connective is any Boolean function with one or two inputs and one output, such as $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT), $\rightarrow$ (implication), and $\leftrightarrow$ (if and only if); and
3. parentheses. Without loss of generality, we assume that there are no redundant parentheses; that is, a formula contains at most one pair of parentheses per Boolean connective.

The satisfiability problem asks whether a given boolean formula is satisfiable. This means that we want to show that given a boolean formula ϕ , does there exist at least one assignment of inputs that makes ϕ evaluate to true?

Formally,

$$\text{SAT} = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable Boolean formula}\}.$$

I want to show satisfiability of boolean formulas is NP-complete.

Proof. I will use the methodology described above.

SAT $\in$ NP: To show that SAT belongs to NP, we show that a certificate consisting of a satisfying assignment for an input formula ϕ can be verified in polynomial time.

The verifying algorithm simply replaces each variable in the formula with its corresponding value and then evaluates the resulting expression. This task can clearly be carried out in polynomial time. If the expression evaluates to 1, then the algorithm has verified that the formula is satisfiable.

SAT is NP-Hard: Let K be an arbitrary boolean circuit. We can transform (or more accurately, transcribe) K into a boolean formula ϕ as follows. First, label each interior wire by a new variable y_j , and label the output wire with a new variable z . The formula ϕ consists of a list of equations, one for each gate, separated by ANDs, followed by a final $\wedge z$.

We claim that the original circuit K is satisfiable if and only if the resulting formula ϕ is satisfiable. It is sufficient to show this using an example.

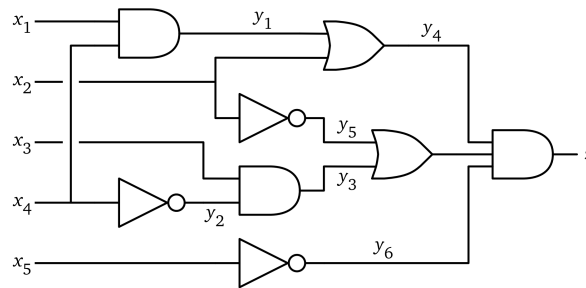


Figure 8.2: Circuit Example for SAT problem

The resulting expression is,

$$\begin{aligned} \phi = & (y_1 \leftrightarrow x_1 \wedge x_2) \wedge (y_2 \leftrightarrow x_2) \wedge (y_3 \leftrightarrow x_3 \wedge y_2) \wedge (y_4 \leftrightarrow y_1 \vee x_2) \wedge (y_5 \leftrightarrow x_3) \wedge \\ & (y_6 \leftrightarrow x_5) \wedge (y_7 \leftrightarrow y_3 \vee y_5) \wedge (z \leftrightarrow y_4 \wedge y_7) \wedge z \end{aligned}$$

We observe that K is satisfiable (in other words, $z = 1$) if and only if ϕ is satisfiable (in other words, $\phi = 1$). To prove this, we will need to prove both directions:

($\Rightarrow$): Given a set of inputs that satisfies the circuit K , this means $z = 1$. Thus, ϕ evaluates to 1. Formally, we can derive a satisfying assignment for the formula ϕ by computing the output of every gate in K .

($\Leftarrow$): Given a satisfying assignment for the formula ϕ , this means $\phi = 1$. This can only occur when $z = 1$, by definition of the truth behaviour of the AND operation. Formally, we can obtain a satisfying input for the circuit by simply ignoring the internal wire variables y_i and the output variable z .

Finally, to show that this transformation is polynomial time. Observe that every gate introduces a constant number of literals. Therefore, the construction takes polynomial time. ■

8.4.2 3-CNF Satisfiability

A Boolean formula is in **conjunctive normal form** (CNF) if it is a conjunction (AND) of several clauses, each of which is a disjunction (OR) of several literals, where each literal is either a variable or its negation. For example,

$$\overbrace{(a \vee b \vee c \vee d)}^{\text{clause}} \wedge (b \vee \neg c \vee \neg d) \wedge (\neg a \vee c \vee d) \wedge (a \vee \neg b).$$

A 3CNF formula is a CNF formula with exactly three literals per clause. The previous example is not a 3CNF formula, since its first clause has four literals and its last clause has only two.

3SAT is the restriction of SAT to 3CNF formulas: given a 3CNF formula, is there an assignment to the variables that makes the formula evaluate to TRUE?

Proof. I will reduce this to circuit SAT.

3CNF-SAT $\in$ NP: To show that 3CNF-SAT belongs to NP, we show that a certificate consisting of a satisfying assignment for an input formula ϕ can be verified in polynomial time. The proof is the same as normal SAT.

SAT is NP-Hard: Given an arbitrary boolean circuit K , we transform K into an equivalent 3CNF formula in several stages. First, we make sure every and and or gate in K has exactly two inputs. If any gate has $k > 2$ inputs, replace it with a binary tree of $k - 1$ binary gates. Then, we transcribe K into a boolean formula ϕ_1 with one clause per gate,

We can then replace the clauses in ϕ_1 with the 3CNF formula to form ϕ . For example, given the statement $w = a \wedge b$, we can deduce the following truth table:

a	b	w	$w \leftrightarrow a \wedge b$
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

The maxterms from the above table is:

$$\begin{aligned}\phi_{\text{IN}} &= (\bar{a} \wedge \bar{b} \wedge w) \vee (\bar{a} \wedge b \wedge w) \vee (a \wedge \bar{b} \wedge w) \vee (a \wedge b \wedge \bar{w}) \\ \therefore \phi_{\text{AND}} &= (a \vee b \vee \bar{w}) \wedge (a \vee \bar{b} \vee \bar{w}) \wedge (\bar{a} \vee b \vee \bar{w}) \wedge (\bar{a} \vee \bar{b} \vee w)\end{aligned}$$

You can do the same with the OR and NAND gates. Thus, the overall ϕ is defined as,

$$\phi = \phi_{\text{AND}} \wedge \phi_{\text{OR}} \wedge \phi_{\text{NAND}}$$

We can now claim the circuit is satisfiable if and only ϕ has a satisfying assignment. This is satisfiable by construction.

Additionally, the construction was polynomial in time as we introduced a constant number of clauses ■

8.4.3 Clique

Clique: A graph where every pair of vertices is connected by an edge.

The decision problem is given any graph G , does G have a clique of size K ?

Proof. **CLIQUE** $\in$ **NP**: First, I need to show that the CLIQUE problem is NP. I can do so easily by checking each pair of vertices to see if they are pairwise connected. This takes polynomial time.

CLIQUE is NP-Hard: I will reduce this problem to the 3CNF-SAT problem. The goal is to show that,

$$\phi \text{ is satisfiable} \Leftrightarrow G \text{ has a clique of a certain size}$$

The reduction algorithm begins with an instance of 3-CNF-SAT. Let $\phi = C_1 \wedge C_2 \wedge C_3 \cdots \wedge C_k$ be boolean formula in 3-CNF with k clauses, each clause C_r has exactly three distinct literals $\ell_1^r, \ell_2^r, \ell_3^r$.

We construct the graph $G = (V, E)$ as follows. For each clause

$$C_r = (\ell_1^r \vee \ell_2^r \vee \ell_3^r)$$

in ϕ , we place a triple of vertices ν_1^r, ν_2^r , and ν_3^r into V . We put an edge between two vertices ν_i^r and ν_j^s if both of the following hold:

- ν_i^r and ν_j^s are in different triples, that is, $r \neq s$, and
- their corresponding literals are consistent, that is, ℓ_i^r is not the negation of ℓ_j^s .

An example is shown below:

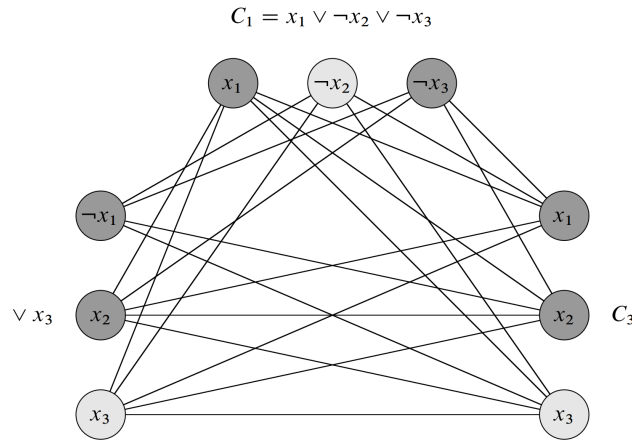


Figure 8.3: Clique Graph

We can easily build this graph from ϕ in polynomial time.

We now prove the correctness of the reduction.

Suppose first that ϕ is satisfiable. Then each clause C_r contains at least one literal that is assigned true. Selecting one such literal from each clause produces a set of k vertices, one from each triple in G . Since no selected literals contradict one another, all corresponding vertices are pairwise connected. Therefore, these vertices form a clique of size k .

Conversely, suppose that G contains a clique of size k . Because no edges connect vertices from the same triple, the clique must contain exactly one vertex from each clause. Assigning the corresponding literals to true satisfies every clause. This assignment is consistent, since conflicting literals are never connected by edges in G . Hence, ϕ is satisfiable.

Thus, ϕ is satisfiable if and only if G has a clique of size k , completing the reduction. ■

8.4.4 Vertex Cover

A **vertex cover** of an undirected graph $G = (V, E)$ is a subset $V' \subseteq V$ such that if $(u, v) \in E$, then $u \in V'$ or $v \in V'$ (or both). That is, it is a set of vertices that all vertices connect to at least one vertex in this set.

The vertex-cover problem is to find a vertex cover of minimum size in a given graph. The decision problem is that we wish to determine whether a graph has a vertex cover of a given size k .

Proof. **VERTEXCOVER** $\in$ **NP**: I can do so easily as given a vertex cover V' with size k , I can check every vertices to see if they are pairwise connected to an element in V . This takes polynomial time.

VERTEXCOVER is NP-Hard: I will reduce this to a CLIQUE problem. First define an **complement graph** as,

Given an undirected graph $G = (V, E)$, we define the **complement** of G as $\overline{G} = (V, \overline{E})$, where

$$\overline{E} = \{(u, v) \mid u, v \in V, u \neq v, (u, v) \notin E\}.$$

In other words, $\overline{G}$ is the graph containing exactly those edges that are not in G .

It can be shown that the vertices that forms a clique in $\overline{G}$ will result in a vertex cover in G . Suppose that G has a clique $V' \subseteq V$ with $|V'| = k$. We claim that $V \setminus V'$ is a vertex cover in $\overline{G}$. Let (u, v) be any edge in $\overline{E}$. Then $(u, v) \notin E$, which implies that at least one of u or v does not belong to V' , since every pair of vertices in V' is connected by an edge of E . This means that at least one of u or v is in $V \setminus V'$, which means that the edge (u, v) is covered by $V \setminus V'$. Since (u, v) was chosen arbitrarily from $\overline{E}$, every edge of $\overline{E}$ is covered by a vertex in $V \setminus V'$. Hence, the set $V \setminus V'$, which has size $|V| - k$, forms a vertex cover for $\overline{G}$.

Conversely, suppose that $\overline{G}$ has a vertex cover $V' \subseteq V$, where $|V'| = |V| - k$. Then, for all $u, v \in V$, if $(u, v) \in \overline{E}$, then $u \in V'$ or $v \in V'$ (or both). The contrapositive of this implication is that for all $u, v \in V$, if $u \notin V'$ and $v \notin V'$, then $(u, v) \in E$. In other words, $V \setminus V'$ is a clique, and it has size $|V| - |V'| = k$.

Finally, it can be observed the inverse graph can be constructed in polynomial time. ■

8.4.5 Traveling Salesman Problem (TSP)

Problem: a salesman must visit n cities. Modeling the problem as a complete graph with n vertices, we can say that the salesman wishes to make a tour, or hamiltonian cycle, visiting each city exactly once and finishing at the city he starts from. The salesman incurs a nonnegative integer cost $c(i, j)$ to travel from city i to city j , and the salesman wishes to make the tour whose total cost is minimum, where the total cost is the sum of the individual costs along the edges of the tour.

Decision Problem: Is there a path that this traveling salesman can take with weight k ?

Proof. **TSP** $\in$ **NP**: I can do so easily as given a path V , I can add the weights along the path to see if they add up to k . This takes polynomial time.

TSP is NP-Hard: I will reduce this to a HAM-CYCLE problem. Given a graph $G = (V, E)$, we construct a graph $G' = (V, E')$ such that $E' = \{(i, j) : i, j \in V, i \neq j\}$. Then, assign weights for G' for every edge $c(u, v)$ such that,

$$c(u, v) = \begin{cases} 0 & \text{if } (u, v) \in G \\ 1 & \text{if } (u, v) \notin G \end{cases}$$

We now show that graph G has a Hamiltonian cycle if and only if graph G' has a tour of cost at most 0.

Suppose that graph G has a Hamiltonian cycle h . Each edge in h belongs to E and thus has cost 0 in G' . Thus, h is a tour in G' with cost 0.

Conversely, suppose that graph G' has a tour h' of cost at most 0. Since the costs of the edges in E' are 0 and 1, the cost of tour h' is exactly 0, and each edge on the tour must have cost 0. Therefore, h' contains only edges in E . We conclude that h' is a Hamiltonian cycle in graph G .

We can also construct G' in polynomial time. ■

8.4.6 Hamiltonian Cycles

A Hamiltonian cycle of an undirected graph $G = (V, E)$ is a simple cycle that contains each vertex in V . We define the hamiltonian-cycle problem as, “Does a graph G have a hamiltonian cycle?”